

Chapter 8

Contents

Chapters

- 8. Flash, EEPROM & Interfacing of memories
 - 8.1 Table Pointer & Operations
 - 8.2 Flash Memory Operations – Reading, Writing/Erasing
 - 8.3 EEPROM Memory Operations – Reading, Writing/Erasing
 - 8.4 External Memory Interface
 - 8.5 Interfacing of External Memories - PIC18C801/PIC18F8720

The concept of memories had been presented in the chapter 1 that shows several memory types, classification and hierarchy. The early programmable devices were mask programmable or used fuses or EPROM, which are one time programmable. PIC18 Series microcontrollers are available with various types of memories like SRAM, EEPROM, Flash memory etc that are reprogrammable.

The Flash memory and EEPROM memories are readable, erasable and writeable. The EEPROM data memories used for higher number of erase and write cycles. Flash program memories are widely used in PIC18F series and are faster compared to EEPROM memories. Both the Flash and EEPROM memories can be read one byte at a time; however, they vary on how data is erased or written. The EEPROM memories allow a write to single memory location whereas a flash memory only allows for write or erase of a group of memory locations known as blocks. The size of the block may vary from one to other based on their capabilities. As presented earlier in chapter 1 memory is divided into blocks consisting of multiple rows and columns and arranged in the form of a matrix with each cell element correspond to a memory location consisting of group of bits.

The selection of memory cell location requires a column and row value which corresponds to a row decoder delay and a column decoder delay etc. In addition, several other delay components exist for every memory access. Assuming the time to access the block of memory is t_b , and additional time to select a specific location in the row is t_r , and time for column selection is t_c , the total time for 64 memory access is $64(t_b + t_r + t_c)$ clock cycles. The parameters t_b , t_r and t_c typically span several clock cycles.

However, some memory structure exists that could automatically select the successive column in that row and produce their data at the output for every clock cycle after the first memory access. Flash memories are of this type where memory devices can input or output a data value for every (single or a multiple) clock cycle after the initial memory block selection there by eliminating the time required for every column selection. The memory access time in such a case would be $64(t_b + t_r) + 64$. So higher the block size, lower is the required memory access time and higher is the overall access speed. Since a block of data can be accessed, Flash memories are faster than EEPROM devices.

The PIC 18 devices have Flash program memory with the program memory word that is 16-bits wide with 21-bit address bus. and EEPROM, SRAM data memory that is 8-bits wide with 16-bit address bus. The following section presents the operation (Read, Write/Erase) of Flash memory. Further, the operations using EEPROM are presented. It is then followed by PIC18s EMI bus and interfacing of external Flash /EEPROM & SRAM memories.

8.1 Table Pointer & Operations

The Flash program memory is not only used for storing program instruction code, but also for storing constants, symbols, tables etc. Since the data is generally stored as tables in flash program memory, the operation of flash memory requires the use of SFR Table registers and instructions that perform operations for accessing table data. The registers and instruction used for accessing memory are given below. The Table 8.1 provides a list of table related instructions.

Registers for Table operations

- Table Pointer Register (TBLPTR) – TBLPTRU, TBLPTRH, TBLPTRL
- Table Latch Register (TABLAT)

Basic Table operations:

- Table Read (TBLRD)
- Table Write (TBLWT)

Table 8.1: Complete list of table related Instructions:

Instruction	Operation	Table Pointer Status
TBLRD*	Table Read	TBLPTR is not modified
TBLRD*-		TBLPTR is decremented after the read
TBLRD*+		TBLPTR is incremented after the read
TBLRD+*		TBLPTR is incremented before the read
TBLWT*	Table Write	TBLPTR is not modified
TBLWT*-		TBLPTR is decremented after the write
TBLWT*+		TBLPTR is incremented after the write
TBLWT+*		TBLPTR is incremented before the write

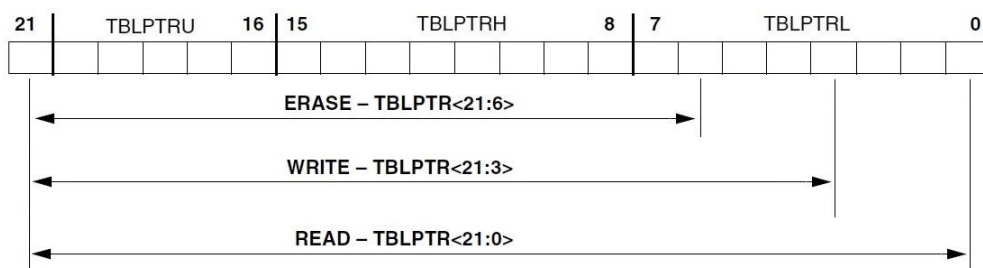


Fig 8.1 Table Pointer boundaries based on operation

The table pointer register is a combination of three 8-bit registers TBLPTRU, TBLPTRH, TBLPTRL. The TBLPTRU register contains the higher 5 bits of the required 21-bits i.e. Bit 21 to Bit 16. The TBLPTRH register contains the next higher 8 bits of the required 21-bits i.e. Bit 15 to Bit 8. The TBLPTRL register contains the lower 8 bits of the required 21-bits i.e. Bit 7 to Bit 0. The lower 21 bits are associated with the memory address used for

providing access to 2M bytes of memory, and the 22nd bit (MSB) provides access to device ID, user ID and configuration bits. The structure of a complete Table Pointer register is shown in Fig 8.1.

The table latch register (TABLAT) holds the 8-bit value that is read from or written to the Flash program memory. During such operations, the data is moved from program memory space to data memory space or vice versa. The TBLPTR register is used as the pointer to hold the address of Flash program memory that is read or written. Every table read or table write instruction makes use of TBLPTR and TABLAT registers and only effect the lower 21 bits. The bit 22 is not affected by read or write operations. TBLPTR is also used for erase operations in addition to read and write. Both the table pointer and table latch are mapped in the SFR address space. The working of TABLAT register during a 16-bit data bus interface for PIC18C801 or PIC18F8720 is presented in external memory interface section.

Program 8.1 Simple 2-Byte Table Read (Program Memory Read)

MOVLW	TABLE_ADDR_U	; Load TBLPTR with base	WREG = 0x10
MOVWF	TBLPTRU	; address of flash program	TBLPTRU = 0x10
MOVLW	TABLE_ADDR_H	; memory	WREG = 0x20
MOVWF	TBLPTRH		TBLPTRH = 0x20
MOVLW	TABLE_ADDR_L		WREG = 0x21
MOVWF	TBLPTRL		TBLPTRL = 0x21
TBLRD*+		; Read low 8-bit data from flash ; program memory into TABLAT ; and increment TBLPTR	TABLAT = [TBLPTR] TBLPTR = TBLPTR + 1
MOVF	TABLAT, 0	: Copy low 8-bit value from ; TABLAT to WREG	WREG = TABLAT
MOVWF	0x50		[0x50] = WREG
TBLRD*+		; Read high 8-bit data from ; flash program memory into ; TABLAT and increment TBLPTR	TABLAT = [TBLPTR] TBLPTR = TBLPTR + 1
MOVF	TABLAT, 0	: Copy high 8-bit value from ; TABLAT to WREG	WREG = TABLAT
MOVWF	0x51		[0x51] = WREG

A simple example of reading program memory is presented below in program 8.1. A graphical representation of a Table read is shown in Fig 8.2. Table accesses are not required to be word aligned as program instruction words. Table block can start and end with an odd or even address. When flash memory locations used for table data are accessed as program instructions, a NOP instruction is executed.

8.2 Flash Memory Operations - Reading & Writing/Erasing

The PIC18F458 has 32K bytes or 16384 single word instruction of Flash memory with address range from 00H to 7FFFH. The PIC18F8720 has 128K bytes or 65536 single word instruction of Flash memory with address ranging from 00H to 01FFFFH.

In PIC18F458 and PIC8720 a flash memory read is performed one byte at a time and write is performed on one block at a time, with a block size of 8 bytes. Erasing of flash memory is also performed one block at time, with a block size of 64 bytes. The table pointer boundaries based on operation is shown in fig 8.1.

Note:

- The program instructions are word aligned and LSB of PC is neglected so that the next instruction is always accessed from an even address.
- There is no restriction for table data and can be starting from an even or odd address.
- When certain flash memory locations are used for table data and are accessed, or when program memory contains invalid instruction codes, a NOP instruction is executed.

The SFR registers used for Flash memory operations are given below.

- Table Latch Register (TABLAT)
- Table Pointer Register (TBLPTR) – TBLPTRU, TBLPTRH, TBLPTRL
- EEPROM Control Register (EECON1 & EECON2)

Read Operation:

A read operation makes use of all the 22 bits of table pointer to determine the location of program memory.

Write Operation:

In a write operation the bits 21 to 3 are used to determine a specific block of program memory. The flash memories have 8 holding registers to store 8-byte data that must be written as a block. The write operation uses the lower 3 least significant bits (LSB) of the table pointer i.e. bits 0 to 2 to select one of the 8-program memory holding registers into which the data from TABLAT register is written.

Note: The Interrupt Flag bit - EEIF in PIR2 register is set when a write operation is complete and must be cleared in software for a normal operation.

Erase Operation:

In an erase operation the higher 16 bits i.e. bit 21 to 6 of the table pointers are used to point to 64-byte block that is erased. The bits 5 to 0 are neglected since a specific location is not required because all locations related to those bits for that block are erased.

EECON1 & EECN2 are the control registers for memory access. EECN1 is an actual physical register but EECN2 is not a physical register. The EECN2 register is used specifically for memory erase and write operations.

A separate procedure is to be followed to erase a bulk memory of more than 64 bytes as it cannot be issued from user code. Additionally, instruction fetch and code execution does not take place during any erase or write operations are completed.

Flash memory is not accessible for read, instruction fetches during write or erase operations. Any write operation is also terminated when an erase operation is issued.

EECON1: EEPROM Control 1 Register Format:

R/W-x	R/W-x	U-0	R/W-0	R/W-x	R/W-0	R/S-0	R/S-0
EEPGD	CFGS	—	FREE	WRERR	WREN	$\overline{WR}$	$\overline{RD}$
bit 7						bit 0	

Bit No	Bit	Description
Bit 7	EEPGD	Flash Program or Data EEPROM Memory Select bit 1 = Access program Flash memory 0 = Access data EEPROM memory
Bit 6	CFGS	Flash Program/Data EE or Configuration Select bit 1 = Access Configuration/calibration registers 0 = Access program Flash or data EEPROM memory
Bit 5	Un implemented	Read as '0'
Bit 4	FREE	Flash Row Erase Enable bit 1 = Erase the program memory row addressed by TBLPTR on the next <u>WR</u> command (cleared by completion of erase operation) 0 = Perform write only
Bit 3	WRERR	Write Error Flag bit 1 = A write operation is prematurely terminated (Any <u>MCLR</u> or any WDT Reset during self-timed programming in normal operation) 0 = The write operation completed Note: When a WRERR occurs, the EEPGD and CFGS bits are not cleared. This allows tracing of the error condition. User may clear this error bit when error occurs and reload the Data & Address registers (EEDATA, EEADR).
Bit 2	WREN	Write Enable bit 1 = Allows write cycles

Bit 1	<u>WR</u>	0 = Inhibits write to the EEPROM or Flash memory Write Control bit 1 = Initiates a data EEPROM erase/write cycle or a program memory erase cycle or write cycle (The operation is self-timed, and the bit is cleared by hardware once write is complete. The <u>WR</u> bit can only be set (not cleared) in software.)
Bit 0	<u>RD</u>	0 = Write cycle to the EEPROM is complete Read Control bit 1 = Initiates an EEPROM read (Read takes one cycle. <u>RD</u> is cleared in hardware. The <u>RD</u> bit can only be set (not cleared) in software. <u>RD</u> bit cannot be set when EEPGD = 1.) 0 = Does not initiate an EEPROM read

Here in the EECON1 register format the R indicates it is Readable bit, similarly W is Writable bit, S is Settable bit, -n is the value at Power on Reset (POR), '1' indicates bit is set, '0' for cleared bit, x if Bit is unknown and U indicates the bit is Unimplemented bit, read as '0'.

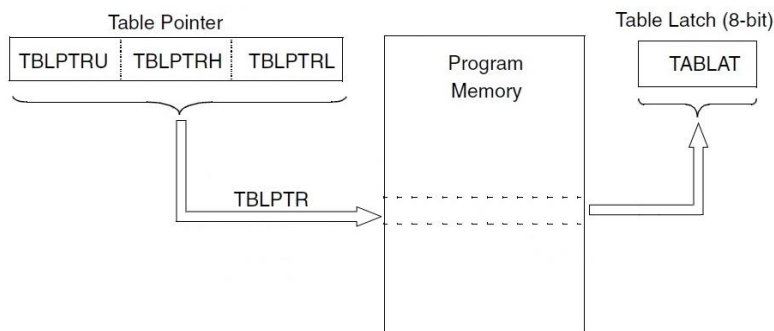


Fig 8.2 Table Read Operation

Source: Microchip Document: PIC18FXX8 Data Sheet - DS41159E

8.2.1 Reading Flash Program Memory

The symbols, constants that are stored in the form of tables in flash program memory are read using TBLRD instruction. As discussed earlier though the instructions stored in program memory are word aligned, the table data need not be word aligned. So, any location in program memory is accessible. TBLRD instruction allows for reading of one byte at a time as 22-bit table pointer points to a location in program memory and the value in the respective location is copied to TABLAT register. The low byte or high byte of a word

are selectable through the LSB bit of the table pointer. Using the indexed addressing mode instructions, the table pointer can be incremented or decremented automatically.

Program 8.2: Simple demonstration example of Table Read

	ORG	30H	; Set Origin 0x30 for Main program
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TBLPTRU,0	; TBLPTRU = 0x00
	MOVLW	0x10	; WREG = 0x10
	MOVWF	TBLPTRH,0	; TBLPTRH = 0x10
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TBLPTRL,0	; TBLPTRL = 0x00
	LFSR	0, 0x100	; FSR0 = 0x100
	MOVLW	0x1C	; WREG = 0x16
	MOVWF	0x01,0	; [0x01] = 0x16
	MOVWF	0x02,0	; [0x02] = 0x16
L1	TBLRD*+		; TABLAT = [0x1000]
	MOVFF	TABLAT, POSTINCO	; [0x100] = TABLAT
			; FSR0 = FSR0 + 1 = 0x101
	DECFSZ	0x02, 1, 0	; [0x02] = [0x02] – 1
	BRA	L1	; Branch to L1 till 16H or 22
			; bytes of data are read from
			; flash program memory
	NOP		; No Operation
HERE	GOTO	HERE	; Loop Here
; Table Data			
	ORG	1000H	; Point to 0x1000 location of
			; program memory. The
			; following table data is saved
			; from this location.
MY_ACII	DB	30H,31H, 32H, 33H, 34H, 35H, 36H, 37H,38H, 39H	; Storing ASCII equivalent ; values of 0-9 in flash program ; memory
MY_MC	DB	"PIC18Fx8"	; Store string "PIC18Fx8"
MY_V1	DB	'M'	; Store string 'M'
MY_V2	DB	'C'	; Store string 'C'
ASCII_REP	DB	'2'	; Store string '2'
DEF_REP	DB		2 ; Store value 2
	NOP		

The Fig 8.3 presents different variations of data, assembler directives and assigned memory locations for the program 8.2.

```

; Table Data
    ORG    1000H

MY_ASCII DB    30H,31H, 32H, 33H, 34H, 35H, 36H, 37H,38H, 39H, 40H
MY_MC    DB    "PIC18Fx8"
MY_V1    DB    "M"
MY_V2    DB    "C"
ASCII_REP    DB '2'
DEF_REP    DB 2
    END

```

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
0E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
100	30	31	32	33	34	35	36	37	38	39	40	00	50	49	43	31	01234567 89@.PIC1
110	38	46	78	38	4D	00	43	00	32	00	02	00	00	00	00	00	@Fxm.C. 2.....
120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Fig 8.3a MY_ASCII with values 30H to 40H followed by other data.

Note: Observe 40H at 100A location and 00H at 100B location.

```

; Table Data
    ORG    1000H

MY_ASCII DB    30H,31H, 32H, 33H, 34H, 35H, 36H, 37H,38H, 39H
MY_MC    DB    "PIC18Fx8"
MY_V1    DB    'M'
MY_V2    DB    'C'
ASCII_REP    DB '2'
DEF_REP    DB 2
    END

```

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
0E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
100	30	31	32	33	34	35	36	37	38	39	50	49	43	31	38	46	01234567 89PIC18F
110	78	38	4D	00	43	00	32	00	02	00	FF	FF	00	00	00	00	x8M.C.2.
120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Fig 8.3b MY_ASCII with values 30H to 39H followed by other data

Note: Observe 50H(P) at 100A location and 49H at 100B location.

```

; Table Data
    ORG    1000H
MY_ASCII DB    30H,31H, 32H, 33H, 34H, 35H, 36H, 37H,38H, 39H
MY_MC    DB    "PIC18Fxx8"
MY_V1    DB    'M'
MY_V2    DB    'C'
ASCII_REP DB '2'
DEF_REP  DB 2
    END

```

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
0E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
100	30	31	32	33	34	35	36	37	38	39	50	49	43	31	38	46	01234567 89PIC18F
110	78	78	38	00	4D	00	43	00	32	00	02	00	00	00	00	00	xx8.M.C. 2.....
120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Memory File Registers Format Hex

Fig 8.3c Continuation of Fig 8.3b &

MY_MC assigned value of "PIC18Fxx8" instead of "PIC18Fx8"

Note: Observe 78H(x) at 110 and 111 locations.

```

; Table Data
    ORG    1000H
P1 CODE_PACK 1000H
MY_ASCII DB    30H,31H, 32H, 33H, 34H, 35H, 36H, 37H,38H, 39H, 40H
MY_MC    DB    "PIC18Fxx8"
MY_V1    DB    'M'
MY_V2    DB    'C'
ASCII_REP DB '2'
DEF_REP  DB 2
    END

```

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
0E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
100	30	31	32	33	34	35	36	37	38	39	40	50	49	43	31	38	01234567 89@PIC18
110	46	78	78	38	4D	43	32	02	FF	FF	FF	FF	00	00	00	00	Fxx8MC2.
120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Memory File Registers Format Hex

Fig 8.3d Combination of Fig 8.3a and Fig 8.3c MY_ASCII having values 30H to 40H and

MY_MC assigned value of "PIC18Fxx8" instead of "PIC18Fx8".

Also uses an Assembler Directive CODE_PACK

Note: Observe 40H and 50H at locations 100A and 100B without a 00H at 100B location.

Similarly, 00H are eliminated at locations after MY_ASCII, MY_MC, MY_V1, MY_V2,

ASCII_REP, DEF_REP arrays with values ending 38H, 4D, 43H, 32, 02H respectively.

The Table Read operation is presented in Fig 8.2. The read operation's procedure is presented in a flow chart in Fig 8.4. An example that demonstrates reading of constant data that was stored as part of the program using assembler directives is presented in program 8.2 with sample data results presented in Fig 8.3.

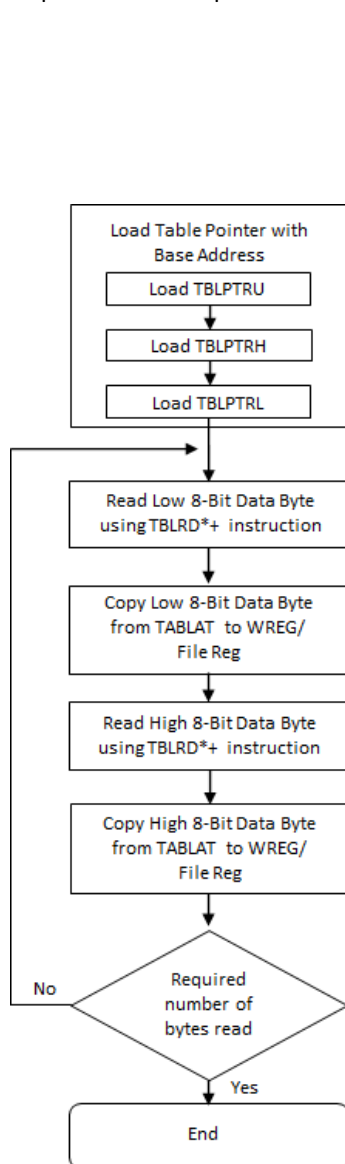


Fig 8.4 Flash Read Operation

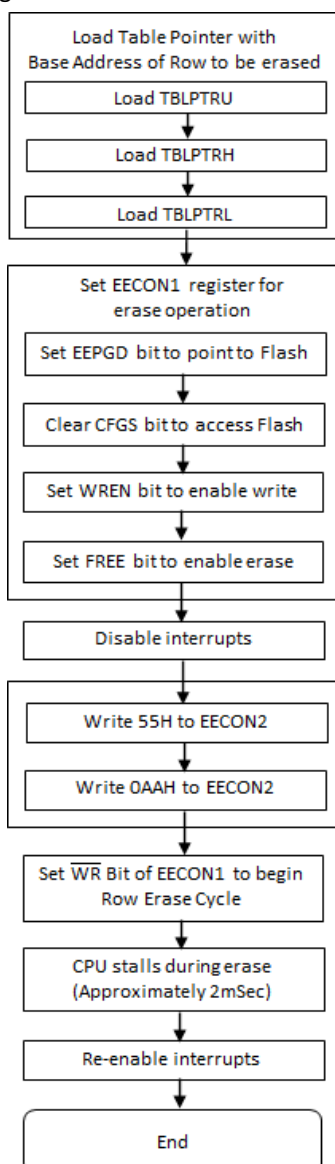


Fig 8.5 Flash Erase Operation

8.2.2 Erasing Flash Memory

The PIC18F458 and PIC18F8720 does not support erasing of a specific word in flash memory but it supports erasing of 32 words or a block of 64-bytes. The erase operation is

performed using the higher 16 bits of address in TBLPTR, i.e the bits 21 to 6 of the table pointer are used to point to 64-byte row block that need to be erased and bits 5 to 0 are ignored. The erase operation's procedure is presented in a flow chart in Fig 8.5.

The EECON1 register is used to control the erase operation. The EEPGD bit must be set and CFGS bit must be cleared to access program Flash memory. The FREE bit & WREN bit must also be set to select erase operation and to allow write cycles using which erase operation is performed. A specific write sequence is to be followed for memory protection during the erase operation. EECON2 is used for the protection purpose, and it need to be written with 0x55 followed by 0xAA before setting the WR bit of EECON1 to begin the row erase cycle.

An erase operation of Flash program memory requires a long write during which program memory read operations - instruction fetch is halted for around 2ms which is controlled through internal timer. A NOP instruction (not shown in erase operation flow chart) may be executed before re-enabling the interrupts. A bulk or block size higher than 64 bytes could only be erased through an external programmer, or through ICSP control.

A simple example of erasing program memory is presented below in program 8.3.

Program 8.3 Simple demonstration example of erasing flash memory row.

MOVLW	TABLE_ADDR_U	; WREG = 0x00	; Load TBLPTR with
MOVWF	TBLPTRU	; TBLPTRU = 0x00	; base/row address
MOVLW	TABLE_ADDR_H	; WREG = 0x20	of
MOVWF	TBLPTRH	; TBLPTRH = 0x20	; flash memory block
MOVLW	TABLE_ADDR_L	; WREG = 0x21	to
MOVWF	TBLPTRL	; TBLPTRL = 0x21	; erase
BSF	EECON1, EEPGD	; Select FLASH memory.	; EECON1 Control
BCF	EECON1, CFGS	; Access FLASH memory	
BSF	EECON1, WREN	; Enable write to memory	
BSF	EECON1, FREE	; Enable Row Erase	
BCF	INTCON, GIE	; Disable Interrupts,	
		; Global Interrupt Disable	
MOVLW	0x55	; WREG = 0x55	; EECON2 Memory
MOVWF	EECON2	; EECON2 = 0x55	; Protection,
MOVLW	0xAA	; WREG = 0xAA	; Disable Spurious
MOVWF	EECON2	; EECON2 = 0xAA	writes
BSF	EECON1, WR	; Start Erase	; EECON1
		; (Stall CPU execution 2mSec)	
NOP		; NOP required for proper	
		; code execution (Delay)	
BSF	INTCON, GIE	; Re-Enable Interrupts,	
		; Global Interrupt Enable	

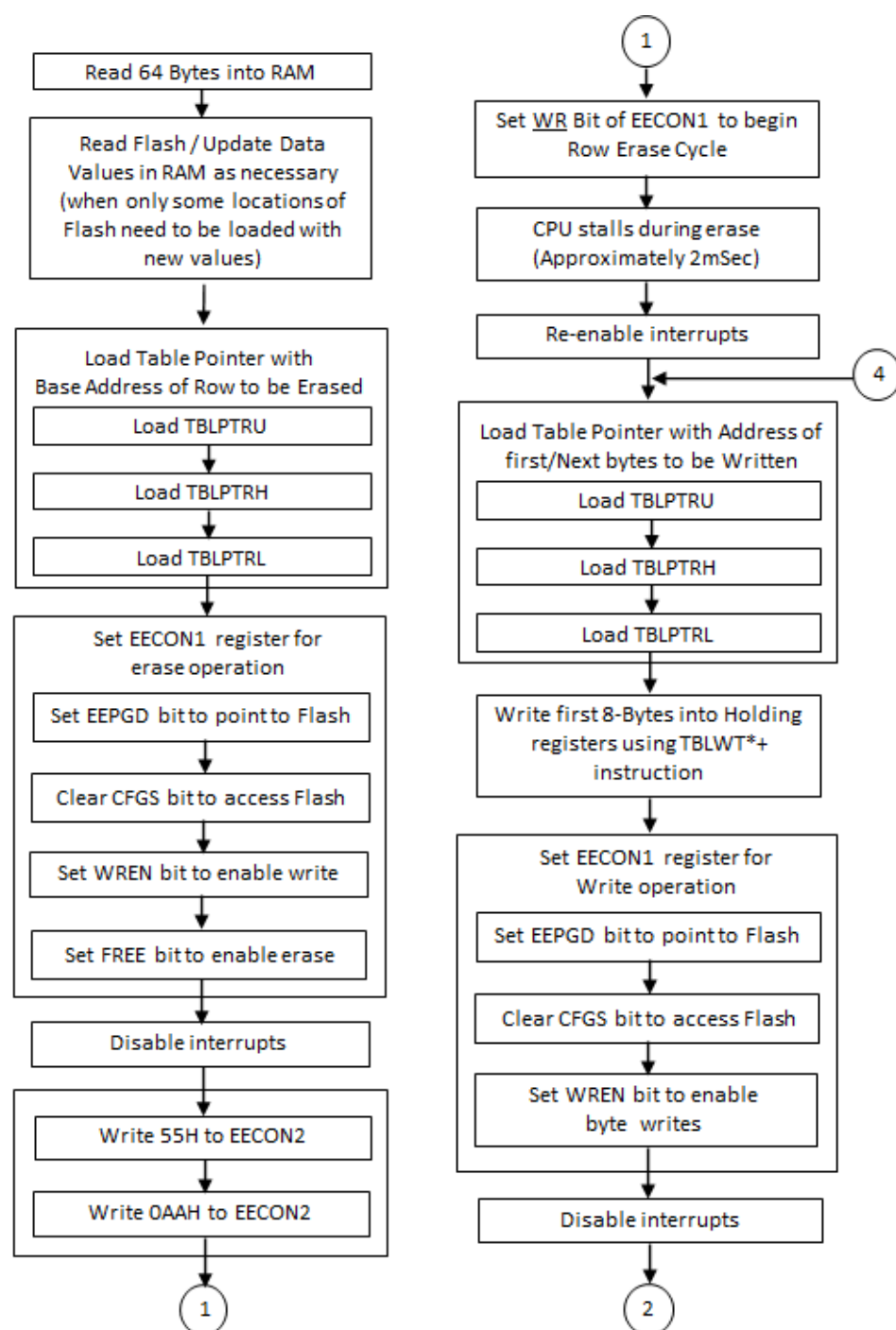


Fig 8.6 (A) Flash Write Operation

(with auto increment TBLPTR) must be used 8 times for storing each data byte into to a specific holding register. The specific holding register is selected using the bits 2 to 0 of the TBLPTR. The write operation that uses holding registers is demonstrated in fig 8.8.

Since only the holding registers are written only a short write is performed. After writing the 8 holding registers, the EECON1 register need to be loaded with appropriate value to start the write process (i.e. the long write). The write operation requires erasing of memory before performing a write. This halts the execution of instructions. The write operation procedure is presented as a flow chart in Fig 8.6.

The completion of a write cycle sets the Interrupt flag bit, EEIF flag in the PIR2 register must be cleared in user's software for operations based on interrupts. A sample program to demonstrate write operation is presented in program 8.4.

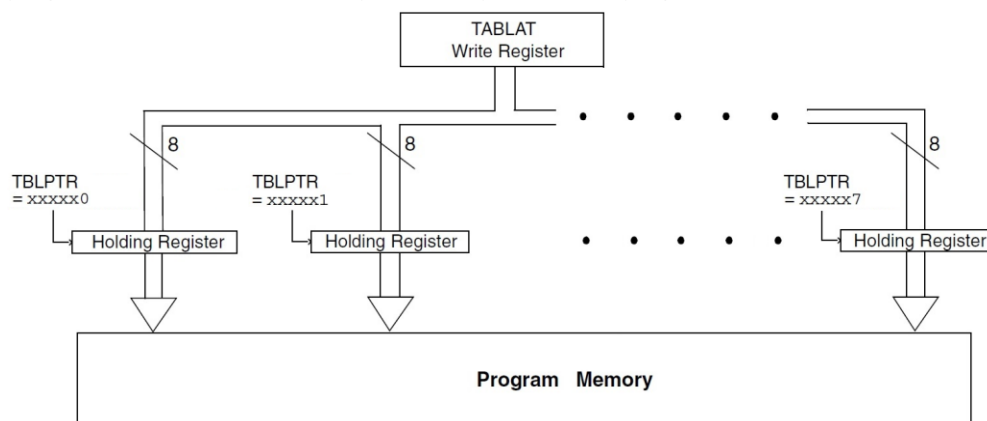


Fig 8.8 Table Write Operation

Write Verify:

One of the steps in standard programming procedures is the verification of memory locations to check whether the data that is written is the data that is stored in the memory. This is widely known as Write Verify and is required when excessive writes are performed that can stress the memory bits near the specification limit.

Additionally, when a write is terminated due to an unplanned event such as loss of power or an unexpected reset a Write Verify & re programming must be performed if required. Whenever a write operation is interrupted during a normal operation due to a MCLR Reset or a WDT Time-Out Reset the WRERR bit in EECON1 is set. Whenever WRERR bit is set, a rewrite operation needs to be performed for data accuracy. A write initiation sequence must be followed like the protection sequence presented earlier in erase operation to protect from spurious writes.

A write operation is performed only after an erase operation. Hence erase procedure of 64 bytes must be performed first. To ensure that required data in the Flash

memory is not lost, the 64 bytes must be read to RAM and updated with required values before a write operation is performed.

Program 8.4 Simple demonstration example of writing to flash memory where 0x100 is starting FSR0 pointer address to copy the table data (also place for modified data) and 8 new values are obtained from 0x210 and updated/copied to address starting from 0x110.

```

; Copy 64-Byte row block that is being erased.
        MOVLW    D'64                ; Row Erase Block Count,
                                        ; WREG = 0x40 = D'64
        MOVWF    0x10, 0            ; [0x10] = 0x40; Count Value
        LFSR     0, 0x100           ; FSR0 = 0x100
                                        ; Data Memory Address from where
                                        ; 64-byte program memory erase
                                        ; table data is going to be stored
        MOVLW    TABLE_ADDR_U     ; Set Table Pointer to base – Row block
                                        ; WREG = 0x00
        MOVWF    TBLPTRU            ; TBLPTRU = 0x00
        MOVLW    TABLE_ADDR_H     ; WREG = 0x10
        MOVWF    TBLPTRH            ; TBLPTRH = 0x10
        MOVLW    TABLE_ADDR_L     ; WREG = 0x00
        MOVWF    TBLPTRL            ; TBLPTRL = 0x00
L1_READ  TBLRD*+                    ; Read Flash memory, get data byte
                                        ; into TABLAT register, Increment
                                        ; TBLPTR
        MOVF     TABLAT, 0           ; Read TABLAT, Copy to WREG
        MOVWF    POSTINC0           ; Copy Flash Table data to RAM
                                        ; Data memory from address
                                        ; 0x100, [FSR0] = WREG
                                        ; FSR0 = FSR0 + 1
        DECFSZ   0x10, 1, 0         ; Decrement file register 0x10,
                                        ; [0x10] = [0x10] – 1, Skip next
                                        ; instruction if Row Erase block count
                                        ; is 0, else go to L1
        BRA      L1_READ            ; Go to L1 63 times. i.e till 64 bytes
                                        ; are copied from Flash to RAM.

; Modify the read 64-Byte row block data if required.
; Modifying 8 bytes starting from 0x110
        MOVLW    D'8                ; Modified Data Count, WREG = 0x08
                                        ; Count to be less than 64
        MOVWF    0x11, 0            ; [0x11] = 0x08; Modified Count Value
        LFSR     0, 0x110           ; FSR0 = 0x110
                                        ; Data Memory Address from where
                                        ; data is being modified. This address
                                        ; is within the 64-byte copied earlier.

```

	LFSR	1, 0x210	; FSR1 = 0x210 ; Data Memory Address from where ; data is being copied to get ; modified data.
L2_MOD	MOVF	POSTINC1, 0	; Read new data, Copy to WREG ; WREG = [FSR1], FSR1 = FSR1 + 1
	MOVWF	POSTINC0, 0	; Copy to setup modified data. ; [FSR0] = WREG, FSR0 = FSR0 + 1
	DECFSZ	0x11,1,0	; Decrement file register 0x11, ; [0x11] = [0x11] – 1, Skip next ; instruction if modified data count is ; 0, else go to L2_MOD
	BRA	L2_MOD	; Go to L2_MOD 7 times. i.e till 8 bytes ; are copied from Ptr 1 to Ptr 0.

; Erasing the 64-Bytes of Flash Memory

	MOVLW	TABLE_ADDR_U	; Set Table Pointer to base – Row block ; WREG = 0x00
	MOVWF	TBLPTRU	; TBLPTRU = 0x00
	MOVLW	TABLE_ADDR_H	; WREG = 0x10
	MOVWF	TBLPTRH	; TBLPTRH = 0x10
	MOVLW	TABLE_ADDR_L	; WREG = 0x00
	MOVWF	TBLPTRL	; TBLPTRL = 0x00
	BSF	EECON1, EEPGD	; Select Flash Memory
	BCF	EECON1, CFGS	; Access Flash Memory
	BSF	EECON1, WREN	; Enable Write to Memory
	BSF	EECON1, FREE	; Enable Row Erase
	BCF	INTCON, GIE	; Disable Interrupts ; Global Interrupt Disable
	MOVLW	0x55	; Memory Protection Sequence ; WREG = 0x55
	MOVWF	EECON2	; EECON2 = 0x55
	MOVLW	0AAH	; WREG = 0xAA
	MOVWF	EECON2	; EECON2 = 0xAA
	BSF	EECON1, WR	; Start Erase ; (Stall CPU Execution 2ms)
	NOP		; No Operation - Delay
	BSF	INTCON, GIE	; Re-Enable Interrupts ; Global Interrupt Enable
	TBLRD*-		; Dummy Read, Decrement TBLPTR ; Make TBLPTR point to base address.

; Writing back the modified 64-Bytes to Flash Memory. 64 bytes is written as follows.
; 1) Write 8-Bytes (1 byte at a time into one holding register) every time,
: i.e 0x21 count

```

; 2) Execute step 1 for a total of 8 times. i.e 0x20 count value
; May write less than 64 bytes, i.e any multiple of 8.
; In that case, step 2 count changes.

        MOVLW    0x8           ; Repeat count – Step 2
        MOVWF    0x20, 0       ; [0x20] = 0x08
        LFSR     0, 0x100      ; FSR0 = 0x100
                                   ; Data Memory Address containing 64
                                   ; byte modified data.
L4_8TIME MOVLW    0x8           ; Number of times (8 times for 8 bytes
                                   ; to 8 holding registers) – Step 1
        MOVWF    0x21, 0       ; [0x21] = 0x08
L3_WRITE MOVF     POSTINC0, 0    ; Get modified data byte from data
                                   ; memory to be written to flash
                                   ; WREG = [FSR0], FSR0 = FSR0+1
        MOVWF    TABLAT        ; TABLAT = WREG
        TBLWT+*                ; Store TABLAT data to holding
                                   ; register, Perform short write
        DECFSZ   0x21, 1, 0     ; Decrement file register 0x21,
                                   ; [0x21] = [0x21] – 1, Skip next
                                   ; instruction if 8 holding registers are
                                   ; filled, i.e when count is = 0, else go to
                                   ; L3_WRITE
        BRA      L3_WRITE      ; Go to L3_WRITE 7 times. i.e till 8
                                   ; bytes are written to 8 holding
                                   ; registers.

; Perform Long Write to Flash Memory.
        BSF      EECON1, EEPGD  ; Select Flash Memory
        BCF      EECON1, CFGS   ; Access Flash Memory
        BSF      EECON1, WREN   ; Enable Write to Memory
        BCF      INTCON, GIE    ; Disable Interrupts
                                   ; Global Interrupt Disable
        MOVLW    0x55           ; Memory Protection Sequence
                                   ; WREG = 0x55
        MOVWF    EECON2        ; EECON2 = 0x55
        MOVLW    0AAH          ; WREG = 0xAA
        MOVWF    EECON2        ; EECON2 = 0xAA
        BSF      EECON1, WR     ; Start Erase (Stall CPU Execution 2ms)
        NOP                        ; No Operation - Delay
        BSF      INTCON, GIE    ; Re-Enable Interrupts
                                   ; Global Interrupt Enable
        DECFSZ   0x20, 1, 0     ; Decrement file register 0x20,
                                   ; [0x20] = [0x20] – 1, Skip next
                                   ; instruction if 64 bytes are written,
                                   ; i.e when count is = 0, else go to

```

			; L4_8TIME
BRA	L4_8TIME		; Go to L4_8TIME 7 times.
			; i.e till 64 bytes are written.
BCF	EECON1, WREN		; Disable Write to Memory

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other Resets
TBLPTRU	—	—	bit 21	Program Memory Table Pointer Upper Byte (TBLPTR<20:16>)					--00 0000	--00 0000
TBPLTRH	Program Memory Table Pointer High Byte (TBLPTR<15:8>)								0000 0000	0000 0000
TBLPTRL	Program Memory Table Pointer Low Byte (TBLPTR<7:0>)								0000 0000	0000 0000
TABLAT	Program Memory Table Latch								0000 0000	0000 0000
INTCON	GIE/GIEH	PEIE/ GIEL	TMR0IE	INTE	RBIE	TMR0IF	INTF	RBIF	0000 000x	0000 000u
EECON2	EEPROM Control Register 2 (not a physical register)								—	—
EECON1	EEPGD	CFGFS	—	FREE	WRERR	WREN	$\overline{WR}$	$\overline{RD}$	xx-0 x000	uu-0 u000
IPR2	—	CMIP	—	EEIP	BCLIP	LVDIP	TMR3IP	ECCP1IP	-1-1 1111	-1-1 1111
PIR2	—	CMIF	—	EEIF	BCLIF	LVDIF	TMR3IF	ECCP1IF	-0-0 0000	-0-0 0000
PIE2	—	CMIE	—	EEIE	BCLIE	LVDIE	TMR3IE	ECCP1IE	-0-0 0000	-0-0 0000

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'.
Shaded cells are not used during Flash/EEPROM access.

Fig 8.9 Registers associated with Program Flash Memory

Source: Microchip Document: PIC18FXX8 Data Sheet - DS41159E

8.3 EEPROM Memory Operations - Reading & Writing/Erasing

The PIC18F458 has 256 bytes of EEPROM memory with address range from 00H to FFH. The EEPROM memory is a different data memory in addition to the PIC18s File registers data memory. The PIC18F458 has 1536 bytes of file register data memory with address range from 00H to 5FFH and F00H to FFFH with 600H to EFFH unused/unallocated. So, a total of 256 + 1536 bytes of data memory.

The PIC18F8720 has 1024 bytes of EEPROM memory with address ranging from 00H to 03FFH. The PIC18F8720 has 3840 bytes of file register data memory with address ranging from 00H to EFFH and F60H to FFFH with F00H to F5FH unused/unallocated. So, a total of 1024 + 3840 bytes of data memory.

The EEPROM data memory can be accessed byte wise and can be read or written similar to a file register data memory and is different from the file register address space. The EEPROM is indirectly addressed through SFR registers. They are

- EEADR – EEPROM Address Register
- EEDATA – EEPROM Data Register
- EECON1 – EEPROM Control Register 1
- EECON2 – EEPROM Control Register 2

The EEADR register holds the address of EEPROM memory and the EEDATA register holds the data value to be written or read from EEPROM memory. EECON1 is a physical control register whose format is presented in the Flash memory section. EECON2 is not a physical register and is used specifically for write operations.

EEPROM memories are used for high erase and write applications. Similar to a flash write, EEPROM writes also follow an “erase before write” process i.e data is erased before being written with new data.

8.3.1 Reading EEPROM Data Memory

EEPROM data memory can be read byte wise and makes use of EECON1 register. The steps to be followed are

- Write EEPROM memory address to EEADR register
- Clear EEPGD bit & CFGS bit in EECON1 register to access EEPROM
- Set the RD bit in EECON1 register to enable read.
- The data would be available in the next clock cycle in EEDATA register until another read/write operation.

A sample read operation of an EEPROM memory is presented in Program 8.5.

Program 8.5 Simple demonstration example of EEPROM Read.

```
LFSR      0,0x100      ; Load FSR with 0x100.; FSR0 = 0x100.  
                        ; To store the data read from EEPROM memory  
                        ; into file reg starting from 0x100
```

	MOVLW	0x08	; Byte Count, get 0x08 bytes read from EEPROM
	MOVWF	0x10,0	; Copy Count to file reg 0x10. [0x10] = 0x08
	MOVF	0x11,0,0	; Get Address of EEPROM from File Reg 0x11 for ; Read. WREG = [0x11]; Value can be anywhere ; from 0x00 – 0xFF
	MOVWF	EEADR	; Set EEPROM base access Address, ; EEADR = [0x11]
	BCF	EECON1,EEPGD	; Select/Point EEPROM DATA memory
	BCS	EECON1,CFG5	; Access EEPROM not Configuration bits
L1_R	BSF	EECON1,RD	; Read EEPROM, Get EEPROM data, ; Copy to EEDATA Reg.; EEDATA = [EEADR]
	MOVF	EEDATA,0	; Copy EEPROM data to WREG. ; WREG = EEDATA
	BCF	EECON1,RD	; Disable Read EEPROM
	MOVWF	POSTINC0,0	; Copy the read EEPROM from WREG to local file ; register pointed by FSR0. [FSR0] = WREG
	DECFSZ	0x10,1,0	; Decrement File Reg Count, Skip next ; instruction ; if 0.
	BRA	L1_R	; Branch to L1_R till required bytes (0x08) are ; read from EEPROM.
	NOP		; No Operation

8.3.2 Writing to EEPROM Data Memory

EEPROM data memory can be written byte wise and makes used of EECON1 register. The steps to be followed are

- Write EEPROM memory address to EEADR register
- Write EEDATA register with the required data to be written to EEPROM
- Clear EEPGD bit & CFG5 bit in EECON1 register to access EEPROM
- Set the WREN bit in EECON1 register to enable write operation.
- Complete the protection sequence
 - Disable Interrupts
 - Move 55H to EECON2 register
 - Move 0AAH to EECON2 register
- Set WR bit in EECON1 to begin write operation
- Enable Interrupts
- After Write cycle is completed EEPROM Write Complete Interrupt Flag bit (EEIF) flag is set. Must be cleared by software for further use.
- Clear WREN bit to disable write operations.

Note: Both WR and WREN cannot be set with the same instruction. The WREN bit is not cleared by hardware. At the completion of the write cycle, the WR bit is cleared in hardware. A sample write operation of an EEPROM memory is presented in Program 8.6.

Program 8.6 Simple demonstration example of EEPROM Write - EEPROM Data Write One byte at a time without interrupt

MOVF	0x11,0,0	; Get Address of EEPROM from File Reg 0x11 for ; Write. WREG = [0x11] ; Value can be anywhere from 0x00 – 0xFF
MOVWF	EEADR	; Set EEPROM base access Address, EEADR = [0x11]
MOVF	0x20,0,0	; Get data from local File Reg 0x20 into WREG ; to write into EEPROM; WREG = [0x20]
MOVWF	EEDATA	; Copy WREG data into EEPROM's EEDATA Reg ; for write, WREG = EEDATA.
BCF	EECON1,EEPGD	; Select/Point EEPROM DATA memory
BCF	EECON1,CFG5	; Access EEPROM not Configuration bits
BSF	EECON1,WREN	; Enable Write
BCF	INTCON,GIE	; Disable interrupts
MOVLW	0x55	; WREG = 0x55
MOVWF	EECON2	; EECON2 = 0x55
MOVLW	0AAh	; WREG = 0xAA
MOVWF	EECON2	; EECON2 = 0xAA
BSF	EECON1,WR	; Write EEPROM with data from EEDATA, ; Copy EEDATA Reg.; [EEADR] = EEDATA
BSF	INTCON,GIE	; Enable interrupts
BCF	EECON1,WREN	; Disable writes on write complete (EEIF set). ; EEIF Is set when write cycle is complete.
NOP		; No Operation
; May perform a Write Verify after completion of Write		

A multi byte write operation of an EEPROM memory without interrupts is presented in Program 8.7 and with interrupt is presented in Program 8.8.

Program 8.7 Simple demonstration example of EEPROM Write - EEPROM Data Write of more than one byte without interrupt

LFSR	0,0x100	; Load FSR with 0x100. FSR0 = 0x100. ; To store data in file reg starting from 0x100 ; into EEPROM.
MOVLW	0x08	; Byte Count, get 0x08 bytes write to EEPROM
MOVWF	0x10,0	; Copy Count to file reg 0x10. [0x10] = 0x08
MOVF	0x11,0,0	; Get Address of EEPROM from File Reg 0x11 ; for Write.; WREG = [0x11]; Value can be

			; anywhere from 0x00 – 0xFF
	MOVWF	EEADR	; Set EEPROM base access Address, ; EEADR = [0x11]
	BCF	EECON1,EEPGD	; Select/Point EEPROM DATA memory
	BCF	EECON1,CFG5	; Access EEPROM not Configuration bits
	BCF	INTCON,GIE	; Disable interrupts
	BSF	EECON1,WREN	; Enable Write
L2_W	MOVF	POSTINC0,0,0	; Get data from local File Reg pointed by FSR0 ; into WREG to write into EEPROM. ; WREG = [FSR0]
	MOVWF	EEDATA	; Copy WREG data into EEPROM's EEDATA Reg ; for write, WREG = EEDATA.
	MOVLW	0x55	; WREG = 0x55
	MOVWF	EECON2	; EECON2 = 0x55
	MOVLW	0AAh	; WREG = 0xAA
	MOVWF	EECON2	; EECON2 = 0xAA
	BSF	EECON1,WR	; Write EEPROM with data from EEDATA, ; Copy EEDATA Reg, [EEADR] = EEDATA
L1_WC	BTFS	EECON1,WR	; Check if Write cycle is complete. (PIR2, EEIF)
	BRA	L1_WC	; if write cycle is not complete go to L1_WC, ; wait till set.
	INCF	EEADR, 1,0	; Increment EEADR to write data in next ; EEPROM location. EEADR = EEADR + 1
	DECFSZ	0x10,1,0	; Decrement File Reg Count, Skip next ; instruction if 0.
	BRA	L2_W	; Branch to L2_W till required bytes (0x08) ; are Written to EEPROM.
	BSF	INTCON,GIE	; Enable interrupts
	BCF	EECON1,WREN	; Disable writes on write complete (EEIF set). ; EEIF Is set when write cycle is complete.
	NOP		; No Operation.
			; May perform a Write Verify after completion of Write

Program 8. Simple demonstration example of EEPROM Write - EEPROM Data Write of more than one byte with interrupt

	ORG	0x00	; This is the address of PC after reset.
	BRA	Main	; Branch to Main, Make PC to go to Main.
; ----			
	ORG	0x08	; This is the address of High Priority Interrupt ; vector
	BRA	ISR_EEP	; Branch to ISR_EEP to execute EEPROM ISR. ; Make PC equal to EEPROM ISR.
; ----			

	ORG	0x30	; This is the address of EEPROM ISR
ISR_EEP	CALL	FL_RD	; Read Flash Memory and copy data to RAM
			; Go and execute FL_RD subroutine
	BCF	PIR2, EEIF	; Clear the EEIF Interrupt flag.
	INCF	EEADR, 1,0	; Increment EEADR to write data in next
			; EEPROM location.
	BSF	0x12,0,0	; New Data availability indicator set
			; indicating data availability to write.
	RETFIE		; Return from Interrupt
; ----			
	ORG	0x40	
FL_RD	TBLRD*+		; Read flash memory. Copy data in location
			; pointed by TBLPTR into TABLAT,
			; TBLAT = [TBLPTR], and increment TBLPTR,
			; TBLPTR = TBLPTR+1
	MOVF	TABLAT, W	; Move the data in TABLAT to WREG.
			; WREG = TABLAT
	MOVWF	INDF0,0	; Move data read from Flash (data from
			; WREG got from TABLAT) to file
			; register pointed by FSR0. [FSR0] = WREG
	MOVWF	PORTB,0	; Send the read Flash data to PORTB
	RETURN		; Return from subroutine.
; ----			
	ORG	0x50	
EWR	BCF	EECON1,EEPGD	; Select/Point EEPROM DATA memory
	BCF	EECON1,CFG5	; Access EEPROM
	BCF	INTCON,GIE	; Disable interrupts
	BSF	EECON1,WREN	; Enable Write
	BCF	0x12,0,0	; New Data availability indicator cleared
			; indicating data Written.
	MOVF	POSTINC0,0,0	; Get data from local File Reg pointed by FSR0
			; into WREG to write into EEPROM.
			; WREG = [FSR0]
	MOVWF	EEDATA	; Copy WREG data into EEPROM's EEDATA
			; Register for write, WREG = EEDATA.
	MOVLW	0x55	; WREG = 0x55
	MOVWF	EECON2	; EECON2 = 0x55
	MOVLW	0AAh	; WREG = 0xAA
	MOVWF	EECON2	; EECON2 = 0xAA
	BSF	EECON1,WR	; Write EEPROM with data from EEDATA,
			; Copy EEDATA Reg.; [EEADR] = EEDATA.
	BSF	INTCON,GIE	; Enable interrupts

	BCF	EECON1,WREN	; Disable Write
	RETURN		
	; ----		
	; Main Program		
	ORG	0x100	
Main	LFSR	0,0x100	; Load FSR with 0x100. FSR0 = 0x100. ; To store Flash data into file reg starting from ; 0x100, read this to store into EEPROM.
	MOVLW	0x08	; Byte Count, get 0x08 bytes write to ; EEPROM
	MOVWF	0x10,0	; Copy Count to file reg 0x10. [0x10] = 0x08
	DECF	0x10,1,0	; Compensate for additional Read. ; [0x10] = 0x08-1 = 0x07
	MOVF	0x11,0,0	; Get Address of EEPROM from File Reg 0x11 ; for Write. WREG = [0x11]; ; Value can be anywhere from 0x00 – 0xFF
	MOVWF	EEADR	; Set EEPROM base access Address, ; EEADR = [0x11]
	MOVLW	0x01	; New Data availability indicator for Write. ; WREG = 0x01; Initially set to 1 ; Value 0x00 (No data ready to Write) or ; Value 0x01 (data ready to Write to EEPROM)
	MOVWF	0x12,0	; [0x12] = WREG = 0x01
	MOVLW	0x00	; Get TBLPTR base address - Upper. ; WREG = 0x00
	MOVWF	TBLPTRU	; Load TBLPTR with the base address (Upper) ; TBLPTRU = 0x00
	MOVLW	0x10	; Get TBLPTR base address – High. ; WREG = 0x10
	MOVWF	TBLPTRH	; Load TBLPTR with the base address (High) ; TBLPTRH = 0x10
	MOVLW	0x00	; Get TBLPTR base address – Lower. ; WREG = 0x00
	MOVWF	TBLPTRL	; Load TBLPTR with the base address (Lower) ; TBLPTRL = 0x00
	BSF	INTCON,GIE	; Enable Global Interrupts
	BSF	INTCON,PEIE	; Enable Peripheral Interrupts
	BSF	PIE2, EEIE	; EEPROM Interrupt enable in PIE2 register.
	BSF	IPR2, EEIP	; Set EEPROM Interrupt as High Priority in ; IPR2 Register
	BCF	PIR2, EEIF	; Clear EEIF flag in PIR2 register.
	CALL	FL_RD	; Read Flash Memory and get a byte. ; Go and execute FL_RD subroutine
L1	NOP		; No Operation. Any other control logic may

			; be here instead of NOP
	BTFSS	0x12,0,0	; Check New Data availability indicator for
			; Write. If 1, skip next and go to CALL EWR
			; instruction to Write the data read from
			; Flash into EEPROM. If 0 go to next
			; instruction i.e BRA L1 to keep checking.
	BRA	L1	; Go to L1, Check New Data availability
			; indicator
	CALL	EWR	; Write available data byte to EEPROM.
			; Go and execute EWR subroutine
	DECFSZ	0x10,1,0	; Decrement File Reg Count, Skip next
			; instruction if count is 0. i.e when required
			; bytes are read & written to EEPROM. If not
			; completed i.e count >0 go to next
			; instruction to repeat Flash Read and
			; EEPROM Write
	BRA	L1	; Go to L1, Check New Data availability
			; indicator
HERE	BRA	Here	; Go to HERE, Infinite loop or
			; Any other control logic may be from here.
	NOP		; No Operation

Write Verify:

Similar to a write verify in a flash memory, a good practice is to verify the validity of written data. A value of 0 may be read in place of a 1 due to leakage or 1 in place of a 0 due to short are some causes of failure.

The WREN bit must be cleared to disable and to ensure spurious writes do not happen. A code protect feature is included with PIC18 devices that disables external read and write operations.

EEPROMs are used for high erase and writes. In cases where writes are not as higher or frequent, a refresh routine may be used to ensure the data is free from leakage and other issues. A sample refresh routine is presented in Program 8.9.

Program 8.9 DATA EEPROM REFRESH ROUTINE

	CLRF	EEADR	; Clear EEADR to start at address 0.
			; EEADR = 0x00.
	BCF	EECON1,EEPGD	; Select/Point EEPROM DATA memory
	BCF	EECON1,CFG5	; Access EEPROM not Configuration bits
	BCF	INTCON,GIE	; Disable interrupts
	BSF	EECON1,WREN	; Enable Write
L1_RW			; Loop to refresh array. Read EEPROM,
			; Write the same to EEPROM
	BSF	EECON1,RD	; Read EEPROMs data in the current address

			; location EEDATA = [EEADR]
	MOVLW	0x55	; WREG = 0x55
	MOVWF	EECON2	; EECON2 = 0x55
	MOVLW	0AAh	; WREG = 0xAA
	MOVWF	EECON2	; EECON2 = 0xAA
	BSF	EECON1,WR	; Set WR bit to begin write, [EEADR] = EEDATA
L2_WC	BTFSF	EECON1,WR	; Check if Write cycle is complete. (PIR2, EEIF)
	BRA	L2_WC	; if write cycle not complete go to L2_WC,
			; wait till set. Instead of using label,
			; can use directive to get current address
			; and decrement by 2 i.e use \$-2
	INCF	EEADR, F	; Increment EEPROM address,
			; EEADR = EEADR + 1,
			; Skip next instruction if EEADR = 0.
	BRA	L1_RW	; Not zero, do it again
	BCF	EECON1,WREN	; Disable writes
	BSF	INTCON,GIE	; Enable interrupts
	NOP		; No Operation.

8.4 External Memory Interface

The PIC18F458 operates exclusively from internal Flash program and RAM data memory in addition to EEPROM data memory. It has no external memory interface and hence no external memory can be interfaced, and the memory capacity cannot be increased.

The PIC18C801 has On-chip RAM (bytes), it however does not have any internal program memory or EEPROM data memory. It has an external memory interface using which Flash, EEPROM, SRAM or other memory can be interfaced. The external memory has address/data and control lines using which the instructions are fetched from external program memory.

The PIC18F8720 provides a combination of memory features available in PIC18F458 and PIC18C801 where it has internal Flash program and SRAM data memory. In addition, it has external memory interface using which external flash program memory, EEPROM or SRAM data memory or other memories are interfaced and provides for increased memory capacity. As it has flexibility of internal and external memories it is provided with configuration modes using which it can be set in different modes. The different Memory Access modes of program memory is presented in Table 8.2. The mode is selected using bits <1:0> i.e. PM1 & PM0 bits of configuration register – CONFIG3L. The available program memory access modes are

- Microprocessor

- Microprocessor with Boot Block
- Microcontroller
- Extended Microcontroller

CONFIG3L – Configuration Byte 3L Format

R/P-1	U-0	U-0	U-0	U-0	U-0	R/P-1	R/P-1
WAIT	—	—	—	—	—	PM1	PM0

bit 7 bit 0

Bit No	Bit	Description
bit 7	WAIT	External Bus Data Wait Enable bit 1 = Wait selections unavailable, device will not wait 0 = Wait programmed by WAIT1 and WAIT0 bits of MEMCOM register (MEMCOM<5:4>)
bit 6-2	Unimplemented	Read as '0'
bit 1-0	PM1:PM0	Processor Data Memory Mode Select bits 11 = Microcontroller mode 10 = Microprocessor mode 01 = Microcontroller with Boot Block mode 00 = Extended Microcontroller mode

Table 8.2: Different Modes for Program Memory Access in PIC18F8720

Operating Mode	Internal Program Memory			External Program Memory		
	Execution From	Table Read From	Table Write To	Execution From	Table Read From	Table Write To
Microprocessor	No Access	No Access	No Access	Yes	Yes	Yes
Microprocessor with Boot Block	Yes	Yes	Yes	Yes	Yes	Yes
Microcontroller	Yes	Yes	Yes	No Access	No Access	No Access
Extended Microcontroller	Yes	Yes	Yes	Yes	Yes	Yes

Microprocessor Mode:

In this mode the PIC18F8720 is similar to a microprocessor where there is no internal memory and external program memory must be interfaced using the external memory interface. The on-chip Flash memory though present is ignored in this mode. All the 21-bit address space of 2M bytes is used for the external program memory.

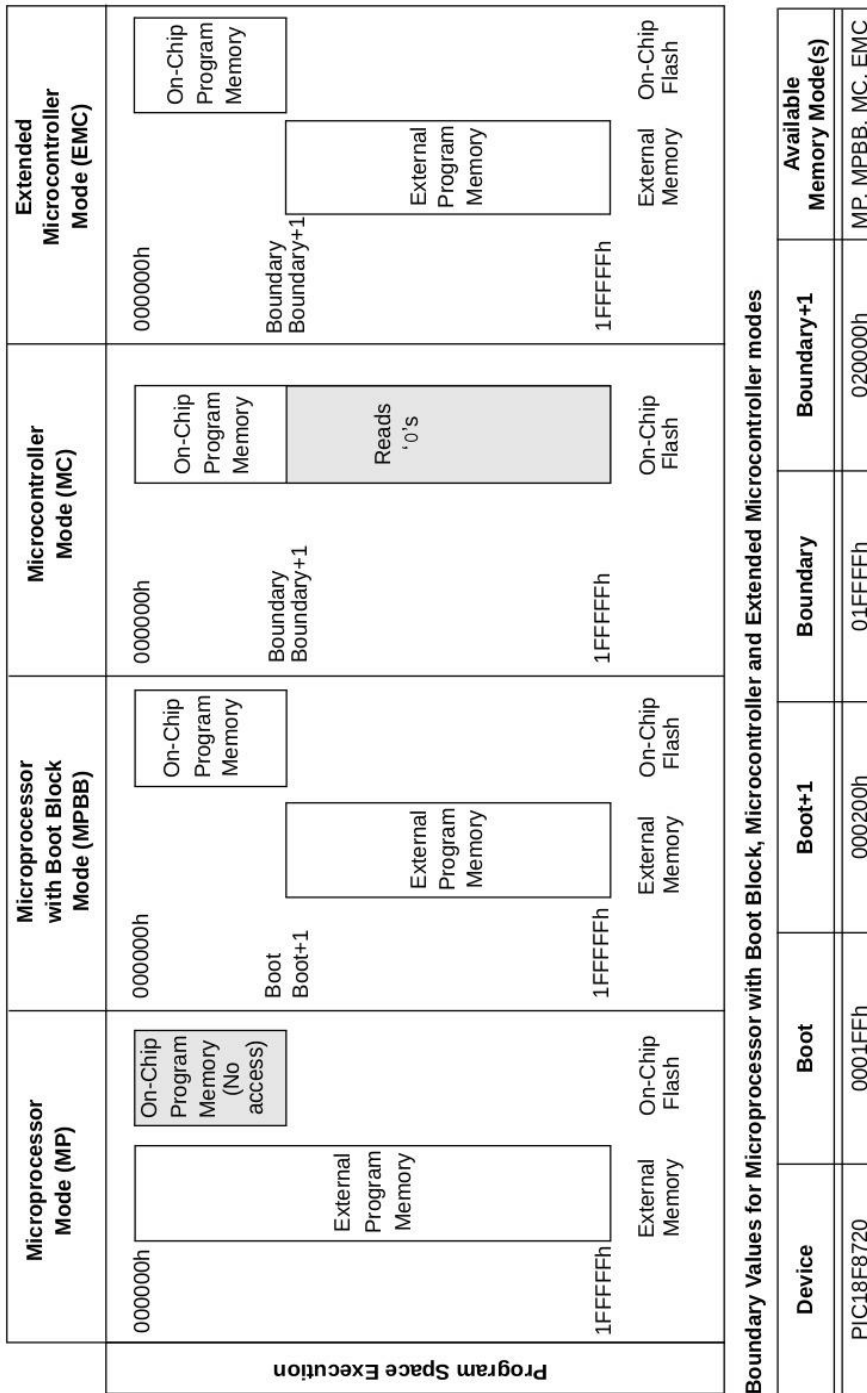


Fig 8.10 Memory Maps for different Program Memory Access Modes

Source: Microchip Document: PIC18FXX20 Data Sheet - DS39609C

Microprocessor with Boot Block Mode:

In this mode the PIC18F8720 can be used to access internal on chip Flash program memory as well as external program memory as a single block of memory. The on-chip Flash memory is typically used for boot programs. The execution switches automatically to internal or external flash memory based on the address location that is accessed.

The on-chip Flash memory is accessible within the address range 000000H to 0001FFH. Any access to addresses above 0001FFH is mapped to the external Flash memory, supporting up to a maximum size of 2MB.

Microcontroller Mode:

In this mode the PIC18F8720 is similar to a regular microcontroller where the required program memory is internal on chip to the microcontroller, external program memory is not required, and hence external memory interface is disabled. The internal 128K bytes or 65536 single word instruction of on chip Flash memory with address ranging from 00H to 01FFFFH is accessible. Any memory access above 01FFFFH results in a read of 0's - NOP instruction.

Extended Microcontroller Mode:

In this mode the PIC18F8720 can be used to access internal on chip Flash program memory as well as external program memory as a single block of memory. The execution switches automatically to internal or external flash memory based on the address location that is accessed.

When address location accessed is in the range of internal 128K bytes (address ranging from 00H to 01FFFFH) or 65536 single word instruction of on chip Flash memory internal Flash memory is accessed. In addition, any memory access above 128K bytes (01FFFFH) up to 2M bytes (01FFFFFFH) results in external flash memory access.

The memory maps for different program memory access modes are presented in Fig 8.10.

MEMCON – Memory Control Register Format

R/W-0	R/W-0	R/W-0	R/W-0	U-0	U-0	R/W-0	R/W-0
EBDIS	PGRM	WAIT1	WAIT0	—	—	WM1	WM0
bit7				bit0			

Bit No	Bit	Description
bit 7	EBDIS	External Bus Disable 1 = External system bus disabled, all external bus drivers are mapped as I/O ports

		0 = External system bus enabled, and I/O ports are disabled
bit 6	PGRM	Unimplemented in PIC18F8720, Read as 0 Program RAM Enable 1 = 512 bytes of internal RAM enabled as internal program memory from location 1FFE00h to 1FFFFFFh, external program memory at these locations are unused. Internal GPR memory from 400h to 5FFh is disabled and returns 00h. 0 = Internal RAM enabled as internal GPR memory from 400h to 5FFh. Program memory from location 1FFE00h to 1FFFFFFh is configured as external program memory.
bit 5-4	WAIT<1:0>	Table Reads and Writes Bus Cycle Wait Count 11 = Table reads and writes will wait 0 TCY 10 = Table reads and writes will wait 1 TCY 01 = Table reads and writes will wait 2 TCY 00 = Table reads and writes will wait 3 TCY
bit 3-2	Unimplemented	Read as '0'
bit 1-0	WM<1:0>	TABLWT Operation with 16-bit Bus 1X = Word Write mode: TABLAT<0> and TABLAT<1> word output, <u>WRH</u> active when TABLAT<1> written 01 = Byte Select mode: TABLAT data copied on both MS and LS Byte, <u>WRH</u> and (<u>UB</u> or <u>LB</u>) will activate 00 = Byte Write mode: TABLAT data copied on both MS and LS Byte, <u>WRH</u> or <u>WRL</u> will activate

External Memory Interface Pins:

The External memory interface in a PIC18F8720 has a total of 28(27) pins while PIC18C801 has up to 38 pins. In PIC18C801 these pins are reserved for external address/data bus functions as external program memory must be interfaced. In PIC18F8720 the pins are multiplexed with I/O port pins on different ports.

The external memory interface i.e the External System Bus is enabled when External Bus Disable - EBDIS bit of MEMCON register is cleared or set to 0. When the External System Bus is enabled, these pins are used for external memory interface. When the EBDIS bit is set to 1 (System Bus disabled) these pins are configured as I/O pins only during the program

execution. The Table 8.3 presents different External memory interface pins for both PIC18C801 and PIC18F8720. The address/data bus pins are presented in Table 8.3a and the control pins of PIC18F8720 are presented in Table 8.3b and pins of PIC18C801 are presented in Table 8.3c where it also presents pins used in 8-bit or 16-bit interfacing modes. The interfacing modes are discussed in the next section.

Table 8.3a: External System Bus – External Memory Bus – External Memory Interface Address/Data Pins in PIC18C801 / PIC18F8720

NAME	PORT	BIT	FUNCTION
RD0/AD0	PORTD	Bit 0	I/O Pin or ESB Address or Data Bit 0 (AD0)
RD1/AD1	PORTD	Bit 1	I/O Pin or ESB Address or Data Bit 1 (AD1)
RD2/AD2	PORTD	Bit 2	I/O Pin or ESB Address or Data Bit 2 (AD2)
RD3/AD3	PORTD	Bit 3	I/O Pin or ESB Address or Data Bit 3 (AD3)
RD4/AD4	PORTD	Bit 4	I/O Pin or ESB Address or Data Bit 4 (AD4)
RD5/AD5	PORTD	Bit 5	I/O Pin or ESB Address or Data Bit 5 (AD5)
RD6/AD6	PORTD	Bit 6	I/O Pin or ESB Address or Data Bit 6 (AD6)
RD7/AD7	PORTD	Bit 7	I/O Pin or ESB Address or Data Bit 7 (AD7)
RE0/AD8	PORTE	Bit 0	I/O Pin or ESB Address or Data Bit 8 (AD8)
RE1/AD9	PORTE	Bit 1	I/O Pin or ESB Address or Data Bit 9 (AD9)
RE2/AD10	PORTE	Bit 2	I/O Pin or ESB Address or Data Bit 10 (AD10)
RE3/AD11	PORTE	Bit 3	I/O Pin or ESB Address or Data Bit 11 (AD11)
RE4/AD12	PORTE	Bit 4	I/O Pin or ESB Address or Data Bit 12 (AD12)
RE5/AD13	PORTE	Bit 5	I/O Pin or ESB Address or Data Bit 13 (AD13)
RE6/AD14	PORTE	Bit 6	I/O Pin or ESB Address or Data Bit 14 (AD14)
RE7/AD15	PORTE	Bit 7	I/O Pin or ESB Address or Data Bit 15 (AD15)
RH0/A16	PORTH	Bit 0	I/O Pin or ESB Address Bit 16
RH1/A17	PORTH	Bit 1	I/O Pin or ESB Address Bit 17
RH2/A18	PORTH	Bit 2	I/O Pin or ESB Address Bit 18
RH3/A19	PORTH	Bit 3	I/O Pin or ESB Address Bit 19

Here I/O pin implies Input or Output Pin and ESB implies External System Bus or External Memory Bus.

Table 8.3b: External System Bus – External Memory Interface Control Pins in PIC18F8720

RJ0/ALE	PORTJ	BIT 0	I/O PIN OR ESB CONTROL PIN - ADDRESS LATCH ENABLE (ALE)
RJ1/OE	PORTJ	Bit 1	I/O Pin or ESB Control Pin - Output Enable (<u>OE</u>)
RJ2/WRL	PORTJ	Bit 2	I/O Pin or ESB Control Pin - Write Low (<u>WRL</u>)

RJ3/ <u>WRH</u>	PORTJ	Bit 3	I/O Pin or ESB Control Pin - Write High (<u>WRH</u>)
RJ4/ <u>BA0</u>	PORTJ	Bit 4	I/O Pin or ESB Control Pin - Byte Address Bit 0 (<u>BA0</u>)
RJ5/ <u>CE</u>	PORTJ	Bit 5	I/O Pin or ESB Control Pin - Chip Enable (<u>CE</u>)
RJ6/ <u>LB</u>	PORTJ	Bit 6	I/O Pin or ESB Control Pin - Lower Byte Enable (<u>LB</u>)
RJ7/ <u>UB</u>	PORTJ	Bit 7	I/O Pin or ESB Control Pin - Upper Byte Enable (<u>UB</u>)

Table 8.3c: External System Bus – External Memory Interface Control Pins in PIC18C801

Name	Port	Bit	Function	8-bit Mux Mode	8-bit De-Mux Mode	16-bit Mode
RG0/ <u>ALE</u>	PORTG	Bit 0	Address Latch Enable (ALE) control pin	Y	Y	Y
RG1/ <u>OE</u>	PORTG	Bit 1	Output Enable (<u>OE</u>) control pin	Y	Y	Y
RG2/ <u>WRL</u>	PORTG	Bit 2	Write Low (<u>WRL</u>) control pin	Y	Y	Y
RG3/ <u>WRH</u>	PORTG	Bit 3	Write High (<u>WRH</u>) control pin	N	N	Y
RG4/ <u>BA0</u>	PORTG	Bit 4	Byte address bit 0	Y	Y	Y
RF3/ <u>CSIO</u>	PORTF	Bit 3	Chip Select I/O (<u>CSIO</u>) (See CS Register)	Y	Y	Y
RF4/ <u>CS2</u> / A16	PORTF	Bit 4	Chip Select 2 (<u>CS2</u>) (See CS Register), Ext Mem Address 16 (A16)	N	Y	Y
RF5/ <u>CS1</u>	PORTF	Bit 5	Chip Select 1 (<u>CS1</u>) (See CS Register)	Y	Y	Y
RF6/ <u>LB</u>	PORTF	Bit 6	Lower Byte Enable (<u>LB</u>) control pin	N	N	Y
RF7/ <u>UB</u>	PORTF	Bit 7	Upper Byte Enable (<u>UB</u>) control pin	N	N	Y
I/O	Any Other Port	Pin	I/O as BYTE/WORD control pin for JEDEC FLASH	N	N	Y

Here the respective pins can be configured as I/O pin or the function mentioned above.

Note:

- While the EDBIS bit is set to 1 i.e. System Bus disabled, if the application program in the device tries to access external memory the pins are switched to external system bus.
- While the executing a program from external memory, if the application program sets the EDBIS bit to 1, the setting of EDBIS bit to 1 is delayed till the program execution is transferred to internal memory after which the pins get configured as I/O ports.

- In the Microprocessor Boot Block mode or Extended Microcontroller mode and accessing internal memory, the external memory interface signals are inactive. The Address and Data lines are tri-stated, while signals such as CE, OE, WRH, WRL, UB, LB are set to '1' while the ALE and BA0 are set to '0'.

Memory Modes:

The external memory interface could be operated in several modes as shown in Table 8.3c and signals used vary according to the mode. The summary of all the modes is presented below.

- 8-bit Mode (Used with 8-Bit Memories)
 - 8-bit Multiplexed
 - 8-bit De-Multiplexed
- 16-bit Mode (Can be used with 8-Bit or 16-Bit Memories)
 - 16-bit Byte Write
 - 16-bit Word Write
 - 16-bit Byte Select

In PIC18C801 the memory mode is selected using bits <6> i.e BW bit of configuration register – CONFIG2L. The bits <7, 5:1> are unimplemented. The bit <6> i.e BW bit of configuration register – CONFIG2L when set to 1 selects 16-bit external bus data width mode and when set to 0 selects the 8-bit mode.

Since PIC18F8720 only allows 16-bit mode, the bit <6> BW bit is not available, is unimplemented along with bit <7, 5:4>. The bits <3:1> and bit <0> are used for other special features not presented here.

CONFIG2L – Configuration Byte 2L Format

U-0	R/P-1	U-0	U-0	R/P-1	R/P-1	R/P-1	R/P-1
—	BW	—	—	BORV1	BORV0	BOREN	<u>PWRTE</u>
bit 7				bit 0			

Bit No	Bit	Description
bit 7	Unimplemented	Read as '0'
bit 6	BW	Unimplemented in PIC18F8720 External Bus Data Width bit 1 = 16-bit external bus mode 0 = 8-bit external bus mode
bit 5-4	Unimplemented	Read as '0'
bit 3-2	BORV1:BORV0	Unimplemented in PIC18C801 Brown-out Reset Voltage bits

		11 = VBOR set to 2.5V
		10 = VBOR set to 2.7V
		01 = VBOR set to 4.2V
		00 = VBOR set to 4.5V
bit 1	BOREN	Unimplemented in PIC18C801
		Brown-out Reset Enable bit
		1 = Brown-out Reset enabled
		0 = Brown-out Reset disabled
bit 0	<u>PWRTEN</u>	Power-up Timer Enable bit
		1 = PWRT disabled
		0 = PWRT enabled

8-Bit Modes:

This mode is used with 8-bit data bus memories. PIC18C801 device uses BA0 in all 8-bit modes where the LSB byte value is changed in a portion of instruction cycle, allowing for two bytes to be sequentially fetched in one instruction cycle. Since two accesses are being done in one instruction cycle, the memory access rate should be twice the instruction cycle.

Multiplexed Mode:

In Multiplexed mode data is transferred on same shared 8-bit lower data bus on Port D. It uses control signals such as ALE, BA0, CS1, OE & WRL. Multiplexed mode applies only to PIC18C601.

De-Multiplexed Mode:

In De-Multiplexed mode data is transferred on a dedicated 8-bit data bus using Port J and address is available during the complete instruction cycle on Port D & E. It uses control signals such as BA0, CS1, OE & WRL. The latch & ALE are not required, left unconnected as address is continuously available. De-Multiplexed mode applies only to PIC18C801.

16-Bit Modes:

This mode is used with 8-bit or 16-bit data bus memories. The memory and External Memory Interface have the capability to access 16-bits. The WM<1:0> bits in MEMCON register are used to determine the type of 16-bit mode connections.

16-bit Byte Write:

This mode is used to connect two separate 8-bit memories to allow for a 16-bit operation. This is typically used with EEPROM and Flash Memories and allows table even writes to byte-wide memories. In this mode data memory is interleaved as odd bank and even bank and the data is transferred on shared 16-bit data bus on Port D & Port E.

For a TBLWT, the same 8-bit table latch data TABLAT is transferred on both the high 8-bit data bus Port E and low 8-bit data bus Port D. The control signals WRH or WRL are strobed based on LSB of TBLPTR to store into respective memory location. PIC18F8720 uses other control signals such as ALE, CE, OE. This approach doesn't have byte selection capability

16-bit Word Write:

This mode is used to connect word-wide 16-bit memories to allow for a 16-bit operation. This is typically used with EEPROM and Flash Memories and allows opcode fetches and table reads from all 16-bit memories and table writes any word-wide external memories.

For a TABWT operation when TBLPTR<0> = 0 i.e for an EVEN address the data from Table Latch register – TABLAT is stored into a holding latch, the external Address/Data bus is tri-stated during the data part of the bus cycle without a write signal. During the next TABWT operation when TBLPTR<0> = 1 i.e for an ODD address the data from Table Latch register – TABLAT is transferred on the high address/data bus AD8-AD15 of PORT E, and the holding latch value is transferred on low address/data bus AD0-AD7 of PORT D. The control signal WRH is used as strobe to store data into the word memory location. The WRL pin is not used. The PIC18F8720 uses other control signals such as ALE, CE, OE. The TABWT operation using such a way requires 16-bit or pairs of bytes to write to a word location.

16-bit Byte Select:

This mode is used to connect word-wide 16-bit memories to allow for a 16-bit operation. This is typically used with SRAM and Flash Memories. It allows table write operations to 16-bit word-wide external memories with byte selection capability.

For a TABWT operation, the same 8-bit table latch data TABLAT is transferred on both the high 8-bit data bus Port E and low 8-bit data bus Port D. The control signal WRH is used as strobe during each write cycle to store data into respective memory location. The appropriate control signals BAO or UB, LB signals could select the appropriate byte to write based on the TBLPTR<0> register. The PIC18F8720 uses other control signals such as ALE, CE, OE.

The JEDEC standard Flash memories require using an I/O pin to select Byte/Word and the BAO signal for byte address selection. The JEDEC SRAM memories use UB or LB signals for byte selection.

External Memory Interface Pins - Description:

The section 6.6 presented memory interfacing signals for Intel's 8086 microprocessor, and the section 6.7 presented several memory interfacing concepts. The interfacing pins of Table 6.12 has been re-presented here for reference. The PIC18 devices also has some similar signals for external memory and I/O interface.

Table 6.12: Interfacing pins of an 8086 microprocessor

Signal Name	Description
AD0-AD15	Address/Data Lines
A0-A15	Address Lines
D0-D15	Data Lines
ALE	Address Latch Enable
<u>DEN</u>	Data Enable
<u>DT/R</u>	Data Transmit or Data Receive
<u>M/IO</u>	Memory or IO - Input Output
<u>RD</u>	Read
<u>WR</u>	Write
BHE	Bus High Enable
A0	LSB Bit of Address Bus
A1	LSB+1 Bit of Address Bus
RDY	Ready
INTR	Interrupt Request
<u>INTA</u>	Interrupt Acknowledge
HOLD	Hold
HLDA	Hold Acknowledge

AD0-AD15: These are time multiplexed bus lines for transferring Address or Data. During the initial phase of instruction cycle Address is transferred on these lines, denoted as A0 to A15 and later Data is transferred on these lines, denoted as D0 to D15.

The PIC18 devices implement a 21-bit internal addressing scheme for accessing program memory. However, the external memory interface bus contains only a 20-bit address bus denoted as AD0-AD15, A16-A19 i.e $A_{\langle 19:0 \rangle}$. Similar to how LSB of a Program Counter is neglected as program memory access is word aligned, the ESB bus also provides only a 20-Bit bus neglecting the LSB bit. I.e $A_{\langle 20:1 \rangle}$ of internal memory address bus is linked to ESB bus $A_{\langle 19:0 \rangle}$. The LSB when required is provided through BA0 signal pin & used in some memory access modes.

ALE - Address Latch Enable: The address value is only available on the Address lines A0-A15 during the initial phase of the instruction cycle. Since data is transferred on these lines later, the address value must be stored in the latches to keep the memory block active and select

the desired location. The ALE signal acts as a strobe enable signal to store the value of address in the latches for later use during the data transfer phase. Latches are strobed during an active high signal on this pin.

OE – Output Enable: When the external memory block & its corresponding location is selected, an output enable signal such as OE is asserted to enable the memory’s output latch, allowing the data to be placed on the external system bus for the microcontroller to read. Memory latch is strobed during an active low signal on this pin.

CE – Chip Enable: The CE signal is used as an enable signal for external memory chips in PIC18F8720 as it can be operated in Microcontroller, Extended MC, Microprocessor or MP with boot block mode.

When the device is accessing internal memory external chips must be disabled to save power and isolate the external system bus. Similarly, when device is accessing external memory the required memory chip must be enabled. An active low signal is generated on this pin when external memory is accessed to enable the external chip. It is held high when internal memory is accessed. It is also inactive (high) when device is in sleep mode.

Additional Chip Enable signals may be required when multiple memory devices are mapped using memory mapped I/O. Since there is only one Chip Enable line, additional hardware circuitry is required to generate enable signals for the respective chips.

Chip Selects signals in PIC18C801

The PIC18F8720 has only one Chip Enable and requires additional hardware circuitry when additional chip select signals are required. Devices such as PIC18C801 minimize the need for additional hardware circuitry by providing 3 Chip Select signals as all the memory is external to the device. The 3 Chip Selects signals are

- CSIO
- CS1
- CS2

The Chip selects are generated based on the values stored in two SFR Chip Select Registers – CSEL2 & CSELIO. The format of registers is presented below. The Table 8.4 presents summary of CSEL2, CSELIO registers and Chip Select signals CS2, CS1 & CSIO.

CSEL2 – Chip Select 2 Register Format

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
CSL7	CSL6	CSL5	CSL4	CSL3	CSL2	CSL1	CSL0
bit 7							bit 0

Bit No	Bit	Description
bit 7-0	CSL<7:0>	Chip Select 2 Address Decode bits XXh = All eight bits are compared to the Most Significant bits PC<20:13> of the PC. If PC<20:13> >= CSEL2<7:0> register, up to 8K then the <u>CS2</u> signal is low. If PC<20:13> < CSEL2<7:0>, <u>CS2</u> is high. 00h = <u>CS2</u> is inactive

CSELIO – Chip Select IO Register Format

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
CSIO7	CSIO6	CSIO5	CSIO4	CSIO3	CSIO2	CSIO1	CSIO0
bit7				bit0			

Bit No	Bit	Description
bit 7-0	CSELIO<7:0>	Chip Select IO Address Decode bits XXh = All eight bits are compared to the Most Significant bits PC<20:13> of the PC. If PC<20:13> = CSELIO<7:0>, up to 8K then the <u>CSIO</u> signal is low. Else <u>CSIO</u> is high. 00h = CSIO is inactive

Table 8.4: Summary of CSEL2, CSELIO registers and Chip Select signals CS2, CS1 & CSIO

CSEL2	CSELIO	<u>CS1</u>	<u>CS2</u>	<u>CSIO</u>
00H	00H	Disabled, Configures RF5 as I/O Pin	Disabled, Configures RF4 as I/O Pin	Disabled, Configures RF3 as I/O Pin
00H	> 00H	Enabled, Active Low	Disabled, Configures RF5 as I/O Pin	Enabled, Active Low or High depending on CSELIO
> 00H	00H	Enabled, Active Low	Enabled, Active Low	Disabled, Configures RF3 as I/O Pin
> 00H	> 00H	Enabled, Active Low whenever both <u>CS2</u> and <u>CSIO</u> are high; Also Low for all lower 8K addresses of program memory	Enabled, Active Low or High depending on CSEL2 value	Enabled, Active Low or High depending on CSELIO value

CS1 – Chip Select 1: The state of this signal pin depends on the CSELIO and CSEL2 register. When either of the two registers are programmed with a value other than 00H the CS1 signal becomes active low. If both the registers are programmed with a value of 00H then the CS1 signal pin is disabled and the RF5 pin is configured as I/O. The CS1 signal is always active low for all lower 8K addresses of program memory.

The CS1 signal pin is active low for all address when both the CS2 and CSIO are high. Therefore, the CS1 pin signal value is active low for the addresses below the address that matches PC<20:13> with that of CSEL2 register. i.e. If CSEL2 value is 40H then the CS1 signal is active low in the address range of 000000H to 07FFFFH i.e. up to $(2000H * 40H - 1)$.

CS2 – Chip Select 2: The state of this signal pin depends on the Program Counter & CSEL2 register. When the CSEL2 register is programmed with a value of 00H then the CS2 signal pin is disabled, and RF4 pin is configured as I/O. The CS2 pin signal value is active low for addresses starting from the address that matches PC<20:13> with that of CSEL2 register. i.e. If CSEL2 value is 40H then the CS2 signal is active low for the address above 080000H i.e. from $(2000H * 40H)$.

CSIO - Chip Select I/O: The CSIO signal is typically used for external I/O devices with an address space of 8K. The state of this signal pin depends on the Program Counter & CSELIO register. If the CSELIO value is 00H then the CSIO signal is disabled, and RF3 pin is configured as I/O.

The starting address of I/O is specified through the CSELIO register which is compared with higher 8-bits PC<20:13> of the Program Counter. The CSIO pin signal value is active low if the PC<20:13> matches with that of CSELIO register. i.e. If CSELIO value is A0H then the CSIO signal is active low in the I/O address range of 140000H (i.e. from $2000H * A0H$) to 141FFFFH.

Note:

- If the I/O address range overlaps with address range specified through CSEL2 for Chip Select CS2 signal, then the CSIO pin signal value would be active low, CS2 signal value will be active high in that range, and the corresponding addresses are made available for I/O's.
- In PIC18C801 the last 512 bytes of program memory are assigned to internal program RAM and must not be used for storing instructions or table data as a program memory. The chip select signals are inactive for this address range of last 512 bytes.

PIC18 devices are designed to interface different types of memories including Flash, EEPROM & SRAM with 8-Bit or 16-Bit interfaces. These memories also use different types of

interfacing signals based on the access modes. The following are the pins that may vary from one to other based on the usage.

BA0: The BA0 signal line indicates the LSB of TBLPTR or Program Counter and generally not used externally except in some specific modes.

It is used to select respect address byte in Byte Select mode for JEDEC standard Flash memories. An I/O line is also required to select Byte or Word mode. The BA0 bit is not used in other 16-bit modes in PIC18F8720.

PIC18C801 device uses BA0 in all 8-bit modes where the LSB byte value is changed in a portion of instruction cycle, allowing for two bytes to be sequentially fetched in one instruction cycle. In Multiplexed mode data is transferred on same 8-bit shared lower data bus on Port D, while in De-Multiplexed mode data is transferred on a dedicated 8-bit data bus using Port J.

UB: The JEDEC standard static RAM memories require UB for selecting the upper byte in the Byte Select mode.

LB: The JEDEC standard static RAM memories require LB for selecting the lower byte in the Byte Select mode.

WRH: The write signal WRH is used as a strobe to store the higher byte data into memory.

WRL: The write signal WRL is used as a strobe to store the lower byte data into memory.

The sources for External Memory interface

- Microchip Doc: PIC18C601/801 Data Sheet - DS39541B
- Microchip Doc: PIC18Fxx20 Data Sheet - DS39609C
- Microchip Doc: AN869 - External Memory Interfacing Techniques for PIC18F8XXX - DS00869B

8.5 Interfacing of External Memories - PIC18C801 & PIC18F8720

The PIC18C801 external memory interface supports both 8-bit mode and 16-bit mode. The PIC18F8720 devices external memory interface supports only 16-bit mode, and 8-bit mode is not supported. Some commercially available external memories that can be interfaced with PIC18 is presented in the Table 8.5.

Table 8.5: Sample external memories that can be interfaced to PIC18

Type	PART No	Description	Doc
EEPROM	AT28LV010	1-Megabit (128K x 8) Low Voltage Paged Parallel EEPROM	0395F-PEEPR-08/09
	ATC28C010	1-Megabit (128K x 8) Paged Parallel EEPROM	DS2006331A
Flash	SST39LF SST39VF (010 / 020 / 040)	1 Mbit (128K x 8) 2 Mbit (256K x 8) 4 Mbit (512K x 8) Multi-Purpose Flash	DS20005023E
	SST39SF (010A / 020A / 040A)	1 Mbit (128K x 8) 2 Mbit (256K x 8) 4 Mbit (512K x 8) Multi-Purpose Flash	DS20005022F
	SST39LF SST39VF (200A / 400A / 800A)	2 Mbit (128K x 16) 4 Mbit (256K x 16) 8 Mbit (512K x 16) Multi-Purpose Flash	DS25001A
SRAM	70V9289L 70V9389L	High-Speed 3.3V 64K x 18/16 Synchronous Pipelined Dual Port Static RAM	DSC4856/9
	71024S	CMOS Static RAM 1 Mbit (128K x 8)	
	R1LV0208BSA	2 Mbit (256K x 8) Adv LPSRAM	
	7028L	High-Speed 64K x 16 Dual Port Static RAM	DSC4836/6
	R1LV0216BSB	2 Mbit (128K x 16) Adv LPSRAM	
	R1RP0416DI	4 Mbit (256K x 16) High Speed SRAM Wide Temperature Version	

Example 1: Interface a memory of two 2K bytes of memory using 8-bit mode for PIC18C801. PIC18 has 21-bit address bus.

Sol: In PIC18C801 the memory mode is selected using bits <6> i.e BW bit of configuration register – CONFIG2L. The bit <6> i.e BW bit of configuration register – CONFIG2L when set to 1 selects 16-bit external bus data width mode and when set to 0 selects the 8-bit mode.

For simplicity we start by considering 8-bit data bus 16-bit address as high address bits are assumed to be 0. First list out the possible address values for the 16-bit or the possible combinations as shown below. The possible addresses are 0's at all the address bits to all 1's at all the address bits which makes the address range 0000H to FFFFH.

Now determine the number bits required for a 2K memory or 2K combinations where $1K = 2^{10} = 1024$. i.e 1K requires 10-bits. To get 2K, a multiplication of 2 is required which is achieved by another bit as each additional bit provides a factor of 2. i.e. $2^{11} = 2K$ and hence 11 bits. So, to achieve 2K bytes all the possible 11-bit combinations are required making the address range from 0000H to 07FFH. i.e. A<10:0> address bit as shown below.

A	A	A	A		A	A	A	A		A	A	A	A		A	A	A	A	Address
15	14	13	12		11	10	9	8		7	6	5	4		3	2	1	0	
0	0	0	0		0	0	0	0		0	0	0	0		0	0	0	0	0000H
0	0	0	0		0	1	1	1		1	1	1	1		1	1	1	1	07FFH

The higher bits A11 to A15 may remain zero or set to any specific fixed combination but is fixed to all 0's in the above discussed example. This fixed combination is used as the chip select to access any location of the 2K memory. The chip select is shown in fig 8.11a. The complete interfacing circuit is shown in fig 8.11b. A decoder may be used for the chip select. As an example, a decoder 5:32 with a combination of 00000 may be used to generate the required chip select for the one shown in fig 8.11a. Refer to Table 8.4 for chip select signals.

For the second 2K memory block the A11 bit is to be 1 and A<10:0> will give a combination of 2K memory location addresses.

A	A	A	A		A	A	A	A		A	A	A	A		A	A	A	A	Address
15	14	13	12		11	10	9	8		7	6	5	4		3	2	1	0	
0	0	0	0		1	0	0	0		0	0	0	0		0	0	0	0	0800H
0	0	0	0		1	1	1	1		1	1	1	1		1	1	1	1	0FFFH

For interfacing 2K memory block the chip select required for a 16-bit address bus is given in Fig 8.11a.

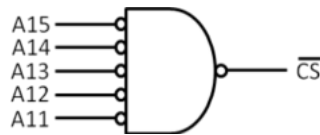


Fig 8.11a Chip Select for 2K Memory

For a 21-bit address bus A15 extends to A20. The external system bus interface has bus lines as A19:A0 which are A<20:1> of internal PC address bits and A0 is available as BA0. Hence the address bus on PIC18 shows A<19:16> & A<15:0>. So here the higher 10 bits i.e A<9:0> ==> PC <10:1> are connected to MA<10:1> of memory and BA0 of PIC18 connected to A0 to complete the total 11 bits required to access 2K Bytes of memory. The data is available on dedicated Data lines on Port J in De-Multiplexed mode PIC18C801.

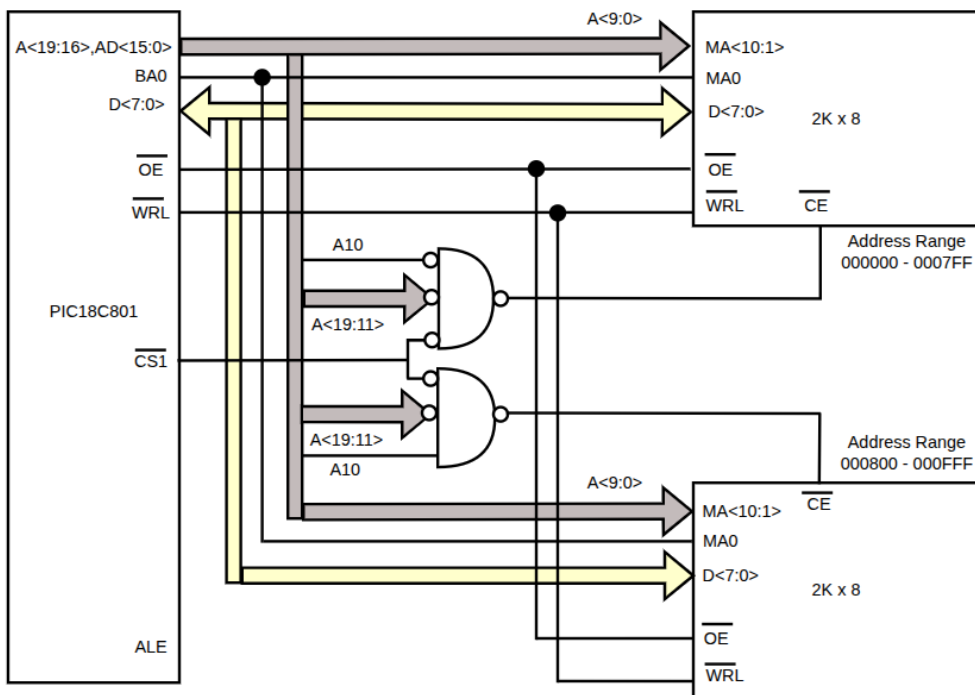


Fig 8.11b Interfacing of two 2K memory – Example 1

Example 2: Interface two Flash memory of size 256K bytes each and two EEPROM memory of size 128K bytes each using PIC18C801 in 16-bit **Byte** write mode and 8K size of I/O devices.

- For two SRAM memory of size 4K bytes each
- For two SRAM memory of size 128K bytes each

Sol: In PIC18C801 the memory mode is selected using bits <6> i.e BW bit of configuration register – CONFIG2L. The bit <6> i.e BW bit of configuration register – CONFIG2L when set to 1 selects 16-bit external bus data width mode and when set to 0 selects the 8-bit mode.

In PIC18C801 the write mode is selected using WM<1:0> i.e bit<1-0> of MEMCON register. A value of 1X selects Word Write mode, a value 01 selects Byte Select mode and a value of 00 select Byte Write mode. For value of 1x give TABLAT0 and TABLAT1 word output, WRH active when TABLAT1 written and value of 01 gives TABLAT data copied on both MS and LS Byte, WRH and (UB or LB) will activate. Here WM<1:0> should be set to 00.

For Flash memory consider SST39SF020A with 2-Megabit (256K x 8) Multi-Purpose Flash with 18 address lines A<17:0> and data lines DQ<7:0>, GND/VSS terminal, Chip Enable CE, Output Enable OE, Write Enable WE and VDD Device power supply.

For EEPROM memory consider AT28C010 with 1-Megabit (128K x 8) Paged Parallel EEPROM with 17 address lines A<16:0> and data lines D<7:0> or I/O<7:0>, GND terminal, Chip Enable CE, Output Enable OE, Write Enable WE and VCC Device power supply.

- A) In case two SRAM memory of size 4K x 8 SRAM, take 12 address lines A<11:0> and data lines D<7:0>, GND/VSS terminal, Chip Enable CE, Output Enable OE, Write Enable WE and VDD Device power supply.
- B) In case two SRAM memory of size 128K bytes each are used then consider 71024S with 1 Mega bit (128K x 8) CMOS Static RAM with 17 address lines A<16:0> and data lines D<7:0>, GND/VSS terminal, Chip Enable CE, Output Enable OE, Write Enable WE and VDD Device power supply.

For a 16-bit mode selection in PIC18C801

Consider Flash Memory

For Flash memory 2 (256K x 8) ==> 512K x 8

$$\Rightarrow 2 * 2^8 * 2^{10} = 2^{19}.$$

I.e 19 Address lines A<18:0> are required to identify each location. The PC<0> i.e A0 is provided separately as BA0 and is not part of the actual 20-bit system address bus A<19:0>. So, keeping A0 or BA0 aside, we require 18 bits to address the 256K byte block and hence A<17:0> address lines are used.

For 256K only 18 are required; 16-bit mode and program memory is accessed as a word. Hence a low – even byte from one 256K byte block and a high – odd byte from another 256K byte block is used, and both the blocks are activated at a time using the same chip select signal. The address values assigned for Flash are 000000 – 07FFFFH. Even addresses in

this range correspond to one block, and odd addresses correspond to the other block, together making up the 512K x 8 Flash memory.

Consider EEPROM Memory

For EEPROM memory 2 (128K x 8) ==> 256K x 8

$$\Rightarrow 2 * 2^7 * 2^{10} = 2^{18}.$$

I.e 18 Address lines A<17:0> are required to identify each location. The PC<0> i.e A0 is provided separately as BA0 and is not part of the actual 20-bit system address bus A<19:0>. So, keeping A0 or BA0 aside, we require 17 bits to address the 128K byte block and hence A<16:0> address lines are used.

For 128K only 17 are required; 16-bit mode is accessed as a word. Hence a low – even byte from one 128K byte block and a high – odd byte from another 128K byte block is used, and both the blocks are activated at a time using the same chip select signal. The next available address is 080000H and hence the address values assigned for EEPROM are 080000H - 0BFFFFH. Even addresses in this range correspond to one block, and odd addresses correspond to the other block, together making up the 256K x 8 EEPROM memory.

Consider SRAM

The RAM is controlled based on the Chip Select 2 CS2 Signal which requires a value to be written to the CSEL2 register. To use the address above 100000H for SRAM using the Chip Select signal, the CSEL2 must be written with a value of 80H to give 2000H * 80H = 100000H. For any address above 100000H, the CS2 signal is active low. If CSELIO is chosen to be in this range, then for the addresses chosen by CSELIO, the CS2 signal is active high and CSIO is active low.

A) Consider SRAM memory two 4K x 8

For SRAM memory 2 (4K x 8) ==> 8K x 8

$$\Rightarrow 2 * 2^2 * 2^{10} = 2^{13}.$$

I.e 13 Address lines A<12:0> are required to identify each location. The PC<0> i.e A0 is provided separately as BA0 and is not part of the actual 20-bit system address bus A<19:0>. So, keeping A0 or BA0 aside, we require 12 bits to address the 4K byte block and hence A<11:0> address lines are used.

For 4K only 12 are required; 16-bit mode is accessed as a word. Hence a low – even byte from one 4K byte block and a high – odd byte from another 4K byte block is used, and both the blocks are activated at a time using the same chip select signal. The next available address is 0C0000H, however the address values assigned for SRAM are 100000H - 101FFFH.

Even addresses in this range correspond to one block, and odd addresses correspond to the other block, together making up the 8K x 8 SRAM memory.

B) Consider SRAM Memory two 128K x 8

For SRAM memory 2 (128K x 8) ==> 256K x 8

$$\Rightarrow 2 * 2^7 * 2^{10} = 2^{18}.$$

I.e 18 Address lines A<17:0> are required to identify each location. The PC<0> i.e A0 is provided separately as BA0 and is not part of the actual 20-bit system address bus A<19:0>. So, keeping A0 or BA0 aside, we require 17 bits to address the 128K byte block and hence A<16:0> address lines are used.

For 128K only 17 are required; 16-bit mode is accessed as a word. Hence a low – even byte from one 128K byte block and a high – odd byte from another 128K byte block is used, and both the blocks are activated at a time using the same chip select signal. The next available address is 0C0000H, however the address values assigned for SRAM are 100000H - 13FFFFH. Even addresses in this range correspond to one block, and odd addresses correspond to the other block, together making up the 256K x 8 EEPROM memory.

Consider I/O devices of fixed 8K size, we divide into two 4K for a 16-bit mode.

For IO 2 (4K x 8) ==> 8K x 8

$$\Rightarrow 2 * 2^2 * 2^{10} = 2^{13}.$$

I.e 13 Address lines A<12:0> are required to identify each location. The PC<0> i.e A0 is provided separately as BA0 and is not part of the actual 20-bit system address bus A<19:0>. So, keeping A0 or BA0 aside, we require 12 bits to address the 4K byte block and hence A<11:0> address lines are used.

For 4K only 12 are required; 16-bit mode is accessed as a word. Hence a low – even byte from one 4K byte block and a high – odd byte from another 4K byte block is used, and both the blocks are activated at a time using the same chip select signal. The next available address is 0C0000H or 140000H, however the address values assigned for SRAM are 140000H - 141FFFH. Even addresses in this range correspond to one block, and odd addresses correspond to the other block, together making up the 8K IO size.

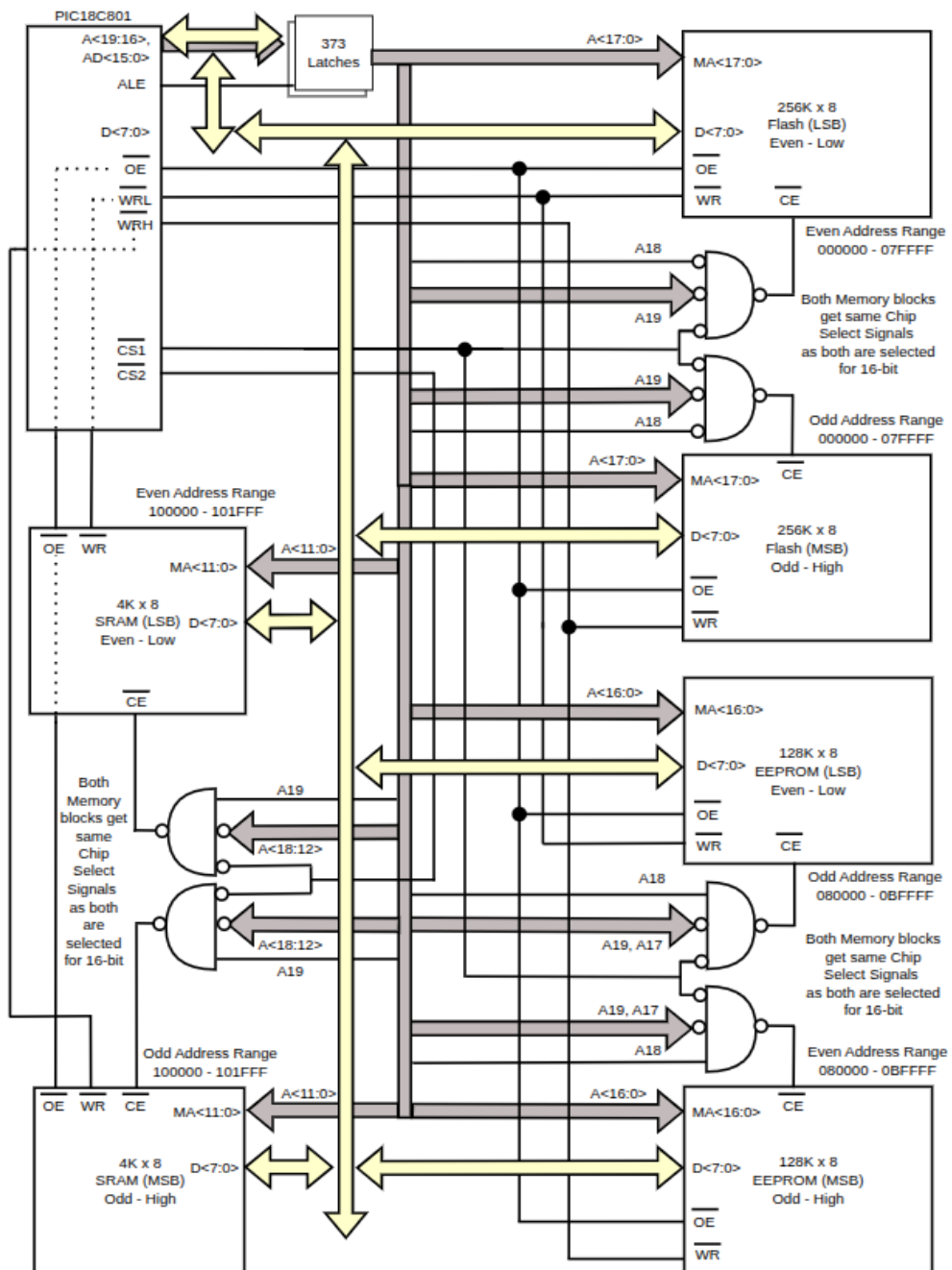


Fig 8.12a PIC18C801 Interfacing with Memory

Here CSEL2 = 80H, CSELIO = A0H. The WR signal only applies to table writes. CSIO and IO block interfacing is not shown due to space constraint. It is similar to SRAM block with signals coming from CSIO instead of CS2 and IO block to get address A<11:0>. However, chip

Enable signals instead of using A<18:12> as 0's may now use A18 --> 0; A17 --> 1, A16, A15, A14, A13, A12 --> 0. i.e. A19 and A17 can be combined to 1 instead of just A19 and keeping the rest (A18, A<16:12>) to 0. It is shown in Fig 8.12b.

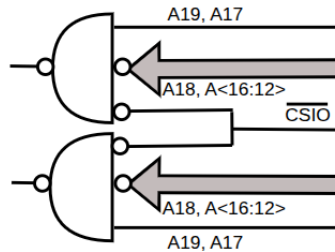


Fig 8.12b PIC18C801 Interfacing – Chip Select signals with IOs

The IO is controlled based on the Chip Select IO CSIO Signal which requires a value to be written to the CSELIO register. To use the address from 140000H for IO using the Chip Select signal, the CSELIO register must be written with a value of A0H to give 2000H * A0H = 140000H. From the address range of 140000H to 141FFFH, the CS2 signal is active high and CSIO is active low. From 142000H, the CS2 is active low as CSELIO is chosen to be in this 140000H to 141FFFH range, which is above 100000H. If CSELIO is chosen as a value of 60H then the IO block would start at 0C0000H and proceed up to 0C1FFFH and CS2 would be active low for all addresses above 100000H. The CSIO signal could be used as a chip select for all IOs.

Example 3: Interface two Flash memory of size 256K bytes each and two EEPROM memory of size 128K bytes each using PIC18F8720 in 16-bit **Byte** write mode and 8K size of I/O devices.

Sol: The PIC18F8720 only allows 16-bit mode and hence bits <6> i.e BW bit of configuration register – CONFIG2L is not required, unimplemented. In PIC18F8720 the write mode is selected using WM<1:0> i.e bit<1-0> of MEMCON register. A value of 1X selects Word Write mode, a value 01 selects Byte Select mode and a value of 00 select Byte Write mode. For value of 1x give TABLAT0 and TABLAT1 word output, WRH active when TABLAT1 written and value of 01 gives TABLAT data copied on both MS and LS Byte, WRH and (UB or LB) will activate. Here WM<1:0> should be set to 00.

PIC18F8720 does not have multiple chip select signals like CS1, CS2 & CSIO and doesn't make use of CSEL2 and CSELIO registers. It only has one Chip Enable pin CE. The interfacing circuit must be modified such that all three signals merge as CE signal pin. CSIO is not shown in Fig 8.12. Hence merge CS1 and CS2 and mark it as CE for PIC18F8720. CSIO may be connected through CE pin.

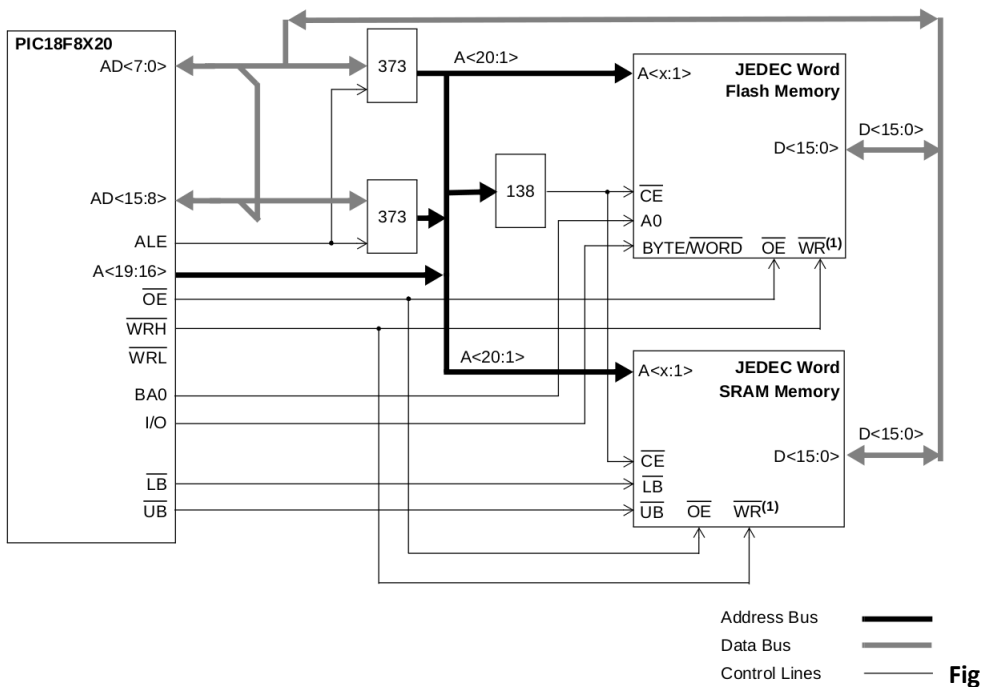


Fig 8.13a PIC18F8720 Interfacing in Byte Select Mode

Source: Microchip Document: PIC18FXX20 Data Sheet - DS39609C

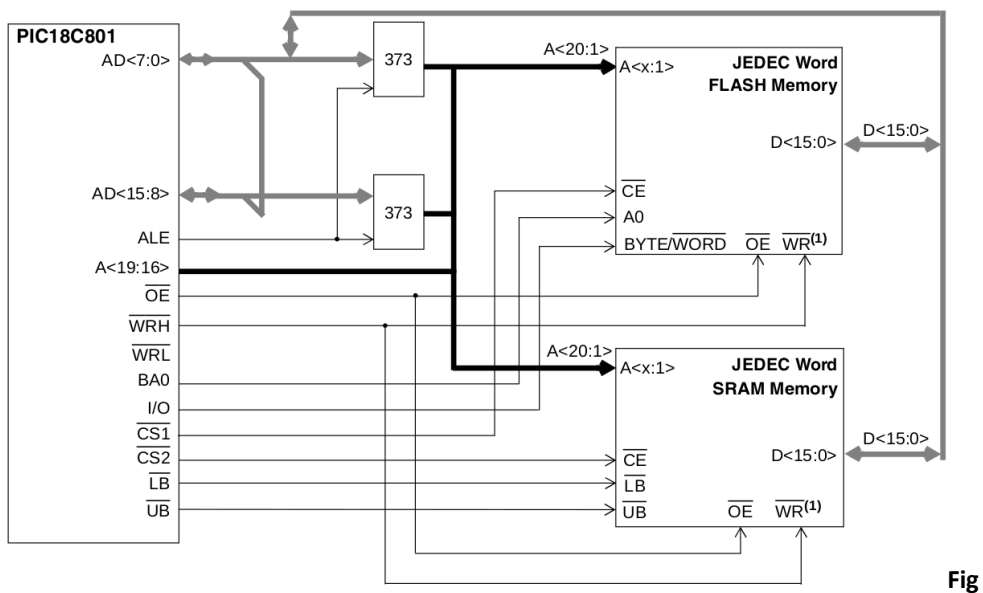


Fig 8.13b PIC18C801 Interfacing in Byte Select Mode

Microchip Document: PIC18C601/801 Data Sheet - DS39541B

Note 1: This signal only applies to Table Writes.

Example 4: Interface in 16-bit **Byte select** mode for PIC18F8720 and PIC18C801.

Sol: In PIC18 the write mode is selected using WM<1:0> i.e bit<1-0> of MEMCON register. A value of 1X selects Word Write mode, a value 01 selects Byte Select mode and a value of 00 select Byte Write mode. For value of 1x give TABLAT0 and TABLAT1 word output, WRH active when TABLAT1 written and value of 01 gives TABLAT data copied on both MS and LS Byte, WRH and (UB or LB) will activate. Here WM<1:0> should be set to 01 to select Byte select mode.

The interfacing circuit of PIC18F8720 is shown in Fig 8.13a and circuit of PIC18C801 is shown in Fig 8.13b.

Example 5: Interface two Flash memory of size 256K bytes each and two EEPROM memory of size 128K bytes each using PIC18C801 in 16-bit **Word** write mode and 8K size of I/O devices.

Sol: In 16-bit Word write mode the interfacing circuit is similar, except that the two odd even blocks combine as one single 16-bit block of 512K x 16 or 256K x 16 or 128K x 16 etc. Considering the above interfacing circuit of Fig 8.12 there will be only 1 block of Flash, EEPROM, SRAM and IO instead of 2 shown with each able to provide a 16-bit value. One noticeable change is that WRL from PIC18 is not used, only WRH is used for that single block.

In PIC18 the write mode is selected using WM<1:0> i.e bit<1-0> of MEMCON register. A value of 1X selects Word Write mode, a value 01 selects Byte Select mode and a value of 00 select Byte Write mode. For value of 1x give TABLAT0 and TABLAT1 word output, WRH active when TABLAT1 written and value of 01 gives TABLAT data copied on both MS and LS Byte, WRH and (UB or LB) will activate. Here WM<1:0> should be set to 10 or 11.